N058 Expo 앱 빌드하고 출시 준비하기_해설서

Chapter01

/강의정리 챕터 01

Expo 앱 빌드하고 출시 준비하기

1. 오늘 강의의 목적

오늘 강의의 핵심은 개발 중인 앱을 실제로 다른 사람도 사용할 수 있는 독립적인 모바일 앱으로 만드는 과정을 이해하는 것입니다.

지금까지 Expo Go를 이용해 휴대폰에서 앱을 실행했다면, 이제는 한 단계 더 나아가야 합니다.

- 내 노트북이 꺼져 있어도 앱이 실행되어야 합니다.
- 친구에게 앱을 전달해서 설치할 수 있어야 합니다.
- 앱을 수정하면 새로운 버전을 만들어 배포할 수 있어야 합니다.
- 최종적으로 구글 플레이스토어와 애플 앱스토어에 출시할 수 있어야 합니다.

이를 위해 배우는 핵심 기술이 **Expo EAS Build(클라우드 앱 빌드)**입니다.

. 중학생도 이해하는 핵심 개념

① Expo Go와 실제 설치 앱의 차이

가장 쉽게 설명하면 Expo Go는 앱을 시험 운전하는 것이고, 실제 빌드 앱은 자동차를 완성해서 출고하는 것입니다.

구분	Expo Go	실제 빌드 앱
실행 방식	Expo Go 안에서 실행	독립적인 앱으로 실행
노트북	개발 서버가 필요	앱 실행에 불필요
코드 수정	개발 서버에서 변경사항 반영	수정 후 새 빌드 또는 업데이트 필요
친구에게 전달	개발 환경 연결 필요	설치 파일 또는 스토어로 전달

구분	Expo Go	실제 빌드 앱
출시	불가능	앱스토어 출시 가능

핵심: Expo Go는 개발과 테스트를 위한 도구이고, 실제 빌드 앱은 사용자에게 배포하기 위한 제품입니다.

② EAS Build란 무엇인가?

오늘의 핵심 기술

EAS = Expo Application Services

EAS Build는 내 컴퓨터 대신 Expo의 클라우드 컴퓨터가 모바일 앱을 만들어 주는 서비스입니다.

예를 들어 다음과 같습니다.

1. 내 컴퓨터에서 앱을 개발합니다.
2. 개발한 코드를 Expo 클라우드에 전달합니다.
3. Expo 클라우드가 필요한 프로그램과 설정을 이용해 앱을 빌드합니다.
4. 완성된 앱 파일을 다운로드합니다.
5. 내 휴대폰에 설치하거나 앱스토어에 제출합니다.

마치 내가 설계도를 보내면 전문 공장에서 완제품을 만들어 주는 것과 같습니다.

A Guide to Expo App Development for Modern Teams - Nerdify Blog

getnerdify.com

③ 앱 파일의 종류

모바일 운영체제에 따라 앱을 설치하는 파일이 다릅니다.

구분	파일 형식	용도
Android	APK	휴대폰에 직접 설치하거나 테스트
Android	AAB	구글 플레이스토어 제출
iOS	IPA	아이폰 앱 설치 및 배포 과정

여기서 중요한 것은 APK와 AAB의 차이입니다.

- APK: 앱을 직접 설치할 수 있는 패키지입니다.
- AAB: 구글 플레이스토어에 제출하는 앱 묶음입니다. 스토어가 기기에 맞는 설치 파일을 생성합니다.

아이폰은 안드로이드와 달리 설치 및 배포에 애플의 서명과 인증 절차가 필요합니다.

④ 앱 빌드와 앱 업데이트의 차이

앱을 처음 만들 때는 빌드가 필요합니다.

그런데 앱을 수정했다고 해서 매번 완전히 새로운 앱을 만들어야 하는 것은 아닙니다.

- 네이티브 기능이나 앱 자체의 구성 변경: 새로운 빌드가 필요할 수 있습니다.
- JavaScript 코드나 화면 구성 변경: EAS Update를 활용하면 조건에 따라 앱 전체를 다시 설치하지 않고 업데이트할 수 있습니다.

즉, EAS Build는 앱을 만드는 공장이고 EAS Update는 이미 설치된 앱에 변경사항을 전달하는 방법이라고 이해하시면 됩니다.

3. 실제 앱 개발 과정과 연결하기

오늘 배운 내용을 AI 친구 에이전트 서비스에 적용해 보겠습니다.

AI Chatbot Mobile App UI/UX Design by Ayon Bepary on Dribbble

dribbble.com

단계	실제 작업	의미
1	React Native + Expo 개발	앱 화면과 기능 제작
2	Expo Go 테스트	개발 중인 앱 확인
3	EAS Build 실행	클라우드에서 앱 파일 생성
4	APK 설치	내 휴대폰에서 독립 실행 확인
5	기능 및 오류 점검	출시 전 품질 확인
6	AAB 또는 iOS 배포 파일 준비	스토어 제출 준비
7	앱스토어 심사 및 출시	일반 사용자에게 서비스 제공

특히 주의할 부분은 앱이 정상적으로 실행되는 것과 서비스가 정상적으로 작동하는 것은 다르다는 점입니다.

예를 들어 AI 친구 앱은 다음을 확인해야 합니다.

- AI 서버와 정상적으로 통신하는가?
- 로그인과 사용자 인증이 정상 작동하는가?
- 대화 내용이 저장되는가?
- 네트워크가 끊겼을 때 오류 처리가 되는가?

- API 키와 비밀번호가 앱에 노출되지 않는가?

4. 앱 출시 전에 준비해야 할 것

앱을 개발하는 것과 앱을 출시하는 것은 다른 문제입니다.

출시하려면 기술적인 준비 외에도 서비스 운영에 필요한 사항이 있습니다.

준비 항목	주요 내용
앱 기본정보	앱 이름, 설명, 버전
앱 아이콘	스마트폰에 표시되는 이미지
스플래시 화면	앱 실행 시 처음 보이는 화면
개인정보 처리방침	개인정보 수집 및 이용 안내
권한 설정	카메라, 마이크, 위치 등
테스트	기기별 기능 및 오류 확인
개발자 계정	Google Play 및 Apple Developer
스토어 등록자료	스크린샷, 설명, 카테고리 등

중요한 점: 앱을 만들었다고 자동으로 스토어에 등록되는 것은 아닙니다. 개발자 계정, 서명, 심사 및 개인정보 관련 준비가 별도로 필요합니다.

5. 비유로 이해하기

앱 개발부터 출시까지를 출판 과정에 비유하면 이해하기 쉽습니다.

앱 개발	책 출판
소스코드 작성	원고 작성
Expo Go 테스트	교정 및 시범 독자 검토
EAS Build	인쇄 및 제본
APK 설치	완성된 책 확인
앱스토어 등록	서점 입점
앱 업데이트	개정판 출판

아무리 좋은 원고라도 책으로 인쇄하고 서점에 입점하지 않으면 독자가 읽을 수 없습니다.

앱도 마찬가지입니다.

6. AI 개발자 및 1인 창업가가 알아야 하는 이유

이번 강의는 단순히 모바일 앱을 만드는 기술을 배우는 시간이 아닙니다.

아이디어를 실제 고객이 사용할 수 있는 제품으로 전환하는 마지막 단계를 배우는 것입니다.

특히 1인 창업가에게 중요한 이유는 세 가지입니다.

1. 개발비 절감: 고성능 Mac을 별도로 준비하지 않고도 EAS 클라우드 빌드를 활용할 수 있습니다. 단, iOS 빌드와 배포에는 애플 계정 및 인증 관련 준비가 필요합니다.
2. 시장 검증: APK를 만들어 지인이나 초기 고객에게 전달하고 실제 반응을 확인할 수 있습니다.
3. 사업 확장: 웹서비스에서 모바일 앱으로 서비스 제공 채널을 확장할 수 있습니다.

7. 나의 프로젝트에 적용하기

현재 준비하고 있는 프로젝트 중 다음과 같은 서비스에 활용할 수 있습니다.

프로젝트	모바일 앱 활용
MSM(My-Story-Maker)	일기 작성, 음성 기록, 나만의 자서전 제작
AI 친구 에이전트	AI와 일상적인 대화 및 개인화된 소통
ASP AI 컨설팅	고객 상담, 프로젝트 진행 상황 확인
Travel Creator	여행 일정 관리, 여행 기록 및 공유

특히 MSM은 모바일 앱과 궁합이 좋습니다.

사람들은 책상 앞에 앉아서만 글을 쓰지 않습니다. 여행 중, 산책 중, 일상생활 중 떠오르는 생각을 바로 기록합니다.

따라서 MSM은 모바일 앱을 통해 글, 사진, 음성을 수집하고 이를 AI가 정리하여 자서전이나 웹소설로 만드는 방향으로 확장할 수 있습니다.

8. 오늘 반드시 기억할 핵심 정리

핵심 용어	기억할 내용
Expo Go	개발 중인 앱을 테스트하는 도구
EAS Build	클라우드에서 실제 모바일 앱을 만드는 서비스
APK	안드로이드 직접 설치용 파일
AAB	구글 플레이스토어 제출용 파일
IPA	iOS 앱 배포에 사용하는 파일
EAS Update	지원되는 변경사항을 앱 전체 재설치 없이 전달하는 기능
앱스토어 심사	일반 사용자에게 앱을 공개하기 위한 절차

9. 오늘의 주관식 퀴즈 답변

Q. 지금 Expo Go로 실행하는 앱과 스토어에서 받은 앱은 무엇이 다를까요?

다음과 같이 정리할 수 있습니다.

1. 노트북을 껐을 때: Expo Go는 개발 서버가 종료되면 개발 중인 앱을 실행할 수 없지만, 빌드된 앱은 독립적으로 실행할 수 있습니다.
2. 코드를 고쳤을 때: Expo Go는 개발 서버를 통해 변경사항을 바로 확인할 수 있지만, 설치된 앱은 새로운 빌드나 EAS Update 등 적절한 배포 절차가 필요합니다.
3. 친구에게 보내고 싶을 때: Expo Go는 개발 환경 연결이 필요하지만, 빌드된 앱은 APK 등의 설치 파일이나 앱스토어를 통해 전달할 수 있습니다.

한 문장으로 정리하면:

Expo Go는 개발자가 앱을 시험해 보는 도구이고, EAS Build로 만든 앱은 노트북과 독립적으로 작동하며 실제 사용자에게 배포할 수 있는 제품입니다.

오늘의 최종 한 줄 정리

개발에서 배포로!

Expo Go로 테스트하는 단계에서 벗어나 EAS Build를 통해 실제 설치 가능한 앱을 만들고, 이를 사용자에게 전달하고 스토어에 출시하는 전체 과정을 이해하는 것이 오늘 강의의 핵심입니다.

Chapter02

/강의정리 챕터 02

앱 파일과 빌드(Build)

1. 오늘 강의의 목적

오늘 강의의 핵심은 내가 작성한 프로그램 코드가 어떻게 실제 스마트폰에서 실행할 수 있는 앱 파일로 변환되는지 이해하는 것입니다.

웹서비스와 모바일 앱은 실행되는 방식이 다릅니다.

- 웹서비스: 인터넷 주소에 접속하면 서버에서 필요한 화면을 받아 실행합니다.

- 모바일 앱: 화면 코드와 이미지, 앱 설정 등을 앱 파일에 담아 스마트폰에 설치하고 실행합니다.

따라서 모바일 앱은 소스 코드만 작성해서는 안 되고, 반드시 빌드(Build)라는 과정을 거쳐야 합니다.

오늘은 다음 세 가지를 이해하는 것이 중요합니다.

1. 앱을 왜 빌드해야 하는가?
2. Android와 iOS는 각각 어떻게 빌드하는가?
3. Expo와 EAS Build는 이 과정을 어떻게 대신하는가?

. 중학생도 이해하는 핵심 개념

① 빌드(Build)가 필요한 이유

빌드는 쉽게 말해 사람이 작성한 설계도를 컴퓨터가 실행할 수 있는 완제품으로 만드는 과정입니다.

예를 들어 건축가가 작성한 설계도만으로 사람이 건물에 들어가 살 수는 없습니다. 실제 건물을 지어야 합니다.

앱도 마찬가지입니다.

PPT - Android Outlook PowerPoint Presentation, free download - ID:5426982

slideserve.com

소스 코드를 스마트폰에서 실행할 수 있는 앱으로 만들려면 세 가지 과정이 필요합니다.

과정	의미	쉬운 비유
컴파일(Compile)	사람이 작성한 코드를 기계가 실행할 수 있는 형태로 변환	외국어를 한국어로 번역
패키징(Packaging)	코드, 이미지, 설정 등을 하나의 앱 파일로 묶음	제품을 상자에 포장
서명(Signing)	개발자가 만든 앱임을 인증	제품에 제조사 인증 도장 찍기

중요한 점: 스마트폰은 개발자가 작성한 소스 코드를 그대로 받아 실행하는 것이 아니라, 운영체제가 요구하는 형식과 인증 조건을 충족한 앱 파일을 설치합니다.

② 네이티브 앱은 어떻게 빌드하는가?

네이티브 앱(Native App)은 Android와 iOS 운영체제에 맞춰 각각의 개발 도구와 언어로 만드는 앱입니다.

Introducing the next generation of Mac - Apple

iOS 앱

- 언어: Swift
- 개발 도구: Xcode
- 실행 환경: Mac
- 결과물: IPA
- 서명: Apple 인증서

Android Studio Giraffe | 2022.3.1 (2023 年 7 月) | Android Developers

Android 앱

- 언어: Kotlin
- 개발 도구: Gradle + Android SDK
- 실행 환경: Windows, Mac, Linux
- 결과물: APK 또는 AAB
- 서명: 개발자 키스토어

두 플랫폼의 차이를 정리하면 다음과 같습니다.

구분	iOS	Android
앱 파일	IPA	APK / AAB
개발 언어	Swift	Kotlin
빌드 도구	Xcode	Gradle
운영체제	Mac 필수	Windows, Mac, Linux
서명	Apple 인증서	개발자 서명 키

여기서 기억해야 할 중요한 사실이 있습니다.

iOS 앱은 Xcode가 Mac에서만 실행되기 때문에 Windows 노트북만으로 직접 빌드할 수 없습니다.

Android는 Windows에서도 빌드할 수 있습니다.

이러한 차이 때문에 두 플랫폼을 네이티브 방식으로 개발하면 각각의 개발과 빌드 과정이 필요합니다.

③ Expo는 어떻게 빌드를 대신하는가?

오늘의 가장 중요한 개념

Expo의 가장 큰 장점은 개발자가 직접 Android와 iOS의 복잡한 빌드 환경을 모두 구축하지 않아도 된다는 것입니다.

Expo는 앱을 크게 두 부분으로 나눕니다.

- JavaScript / TypeScript 부분: 앱의 화면과 동작을 담당합니다.
- 네이티브 부분: 카메라, 알림, 기기 기능 등 스마트폰 운영체제와 연결됩니다.

Complete Guide to Building and Running Expo(React Native) Apps | Madan Bazgai

madanbajgai.com.np

Expo에서는 각각 다음과 같이 처리합니다.

구분	처리 방식	역할
TypeScript 코드	Metro	JavaScript 번들로 묶음
네이티브 코드	EAS Build	Android / iOS용으로 컴파일
개발 테스트	Expo Go	미리 만들어진 네이티브 앱에 내 코드를 연결
실제 앱 제작	EAS Build	독립적으로 설치할 앱 파일 생성

여기서 Metro와 EAS Build를 혼동하면 안 됩니다.

- Metro: 내가 작성한 JavaScript 코드를 실행할 수 있도록 묶는 도구입니다.
- EAS Build: 실제 스마트폰에 설치할 수 있는 앱 파일을 만드는 서비스입니다.

즉, Metro는 코드 묶기, EAS Build는 앱 완제품 만들기입니다.

④ Windows에서도 iOS 앱을 만들 수 있는 이유

일반적으로 iOS 앱을 직접 빌드하려면 Mac과 Xcode가 필요합니다.

그런데 Expo의 EAS Build를 사용하면 이야기가 달라집니다.

Migrating a Native App to React Native? Here’s How Expo Can Help - DEV Community

dev.to

작업 흐름은 다음과 같습니다.

1. Windows 노트북에서 TypeScript 코드를 작성합니다.
2. app.json 등에 앱의 설정을 작성합니다.
3. EAS Build에 빌드를 요청합니다.

4. Expo 클라우드가 필요한 네이티브 프로젝트를 생성합니다.
5. Android는 Linux 기반 빌드 환경에서, iOS는 Mac 기반 빌드 환경에서 컴파일합니다.
6. 서명 작업을 거쳐 최종 앱 파일을 만듭니다.
7. 완성된 앱 파일을 다운로드합니다.

따라서 Windows 노트북만으로도 iOS 앱 빌드가 가능합니다.

이것이 비개발자 1인 창업가에게 매우 유용한 부분입니다.

다만 Apple 개발자 계정과 인증서 등 iOS 배포에 필요한 준비까지 없어지는 것은 아닙니다.

3. 버전 번호와 업데이트 이해하기

앱에는 두 가지 버전 번호가 있습니다.

구분	의미	예시
version	사용자가 보는 앱 버전	1.0.0
versionCode / buildNumber	스토어가 구분하는 빌드 번호	1, 2, 3

예를 들어 AI 친구 앱을 처음 출시한다고 가정하겠습니다.

- 최초 출시: version 1.0.0 / build 1
- 첫 번째 업데이트: version 1.0.1 / build 2
- 두 번째 업데이트: version 1.1.0 / build 3

사용자가 보는 버전은 제품의 변경 정도를 표현하고, 빌드 번호는 스토어가 각각의 앱 파일을 구별하는 데 사용합니다.

스토어에 새 빌드를 제출할 때는 해당 플랫폼의 빌드 번호를 이전보다 크게 만들어야 합니다.

웹서비스와 모바일 앱의 업데이트 차이

이 부분은 실제 서비스 운영에서 매우 중요합니다.

구분	웹서비스	모바일 앱
업데이트	서버에 배포하면 반영	사용자가 앱을 업데이트해야 반영
사용자 환경	일반적으로 최신 화면 사용	구버전 앱이 계속 남을 수 있음
백엔드 변경	API 호환성 관리 필요	구버전 앱과의 호환성 관리가 특히 중요
화면 변경	서버 배포로 반영	새 빌드 또는 EAS Update 필요

예를 들어 AI 친구 앱의 사용자가 1.0 버전을 사용하고 있는데, 내가 2.0 버전을 출시했다고 가정하겠습니다.

모든 사용자가 자동으로 2.0 버전을 사용하는 것은 아닙니다.

일부 사용자는 여전히 1.0 버전을 사용합니다.

그런데 서버의 API 응답 구조를 완전히 바꿔 버리면 구버전 앱은 정상적으로 작동하지 않을 수 있습니다.

따라서 모바일 앱을 운영할 때는 구버전 앱과 새로운 백엔드가 함께 작동할 수 있도록 API 호환성을 유지하는 것이 중요합니다.

4. 어떤 것을 수정하면 다시 빌드해야 할까?

오늘 강의에서 실무적으로 가장 중요한 내용입니다.

모든 코드를 수정할 때마다 앱을 다시 빌드해야 하는 것은 아닙니다.

수정 항목	재빌드 필요 여부
서버의 AI 프롬프트 변경	불필요
백엔드 서버 코드 변경	불필요
데이터베이스 변경	일반적으로 불필요
앱 화면 문구 변경	일반적으로 필요
앱 아이콘 변경	필요
네이티브 기능 추가	필요
네이티브 라이브러리 변경	일반적으로 필요

여기서 주의할 점이 있습니다.

앱 화면 문구처럼 JavaScript 번들에 포함되는 변경은 EAS Update를 통해 다시 빌드하지 않고 배포할 수도 있습니다. 단, 업데이트 대상 앱의 네이티브 런타임과 호환되어야 합니다.

반면 앱의 네이티브 기능이나 앱 자체의 설정이 바뀌면 새로운 빌드가 필요합니다.

핵심 원칙은 앱 파일 안에 들어 있는 것을 수정하는지, 서버에 있는 것을 수정하는지 구분하는 것입니다.

5. 빌드가 실패했을 때 확인할 사항

EAS Build가 실패하면 무작정 코드를 수정하기보다는 빌드 로그를 먼저 확인해야 합니다.

실패 위치	의심할 원인	확인할 것
패키지 설치	라이브러리 충돌	package.json

실패 위치	의심할 원인	확인할 것
네이티브 컴파일	네이티브 라이브러리 오류	라이브러리 버전
앱 설정	설정 오류	app.json
환경변수	필요한 값 누락	EAS 환경변수 설정
서명	인증서 및 키 문제	Expo 계정의 인증 정보

특히 중요한 것은 `.env` 파일입니다.

내 컴퓨터에만 존재하는 `.env` 파일의 값은 클라우드 빌드 서버에 자동으로 전달되지 않습니다.

EAS Build에서 필요한 환경변수는 EAS 환경변수 기능을 이용해 별도로 설정해야 합니다.

또한 API 키처럼 비밀로 관리해야 하는 값은 앱에 포함하지 않는 것이 원칙입니다. 모바일 앱에 포함된 값은 사용자가 분석을 통해 알아낼 가능성이 있기 때문입니다.

6. AI 개발자 및 1인 창업가가 알아야 하는 이유

이번 강의는 단순히 앱을 만드는 방법보다 앱을 지속적으로 운영하는 구조를 이해하는 것이 중요합니다.

1. 개발 비용 절감: Mac을 직접 구매하지 않고도 Expo 클라우드에서 iOS 앱을 빌드할 수 있습니다.
2. 빠른 제품 출시: 앱 개발과 빌드 환경 구축에 들어가는 시간을 줄일 수 있습니다.
3. 서비스 운영 안정성: 구버전 앱과 최신 백엔드 사이의 호환성을 고려할 수 있습니다.
4. 유지보수 비용 절감: 서버 변경과 앱 변경을 구분해 불필요한 재빌드를 줄일 수 있습니다.
5. 사업 확장: 웹서비스로 시작한 AI 서비스를 모바일 앱으로 확장할 수 있습니다.

특히 비개발자 창업가는 코드를 직접 모두 작성하는 것보다 개발 구조를 이해하고, 필요한 작업을 AI 개발 도구에 정확하게 지시할 수 있는 능력이 중요합니다.

7. 나의 프로젝트에 적용하기

현재 진행하거나 준비 중인 프로젝트를 기준으로 적용해 보겠습니다.

프로젝트	빌드 및 업데이트 전략
MSM	모바일 글쓰기 화면은 앱에 구현하고 AI 자서전 생성 기능은 서버에서 운영
AI 친구 에이전트	대화 화면은 앱, AI 프롬프트와 에이전트 로직은 백엔드에서 관리
ASP 컨설팅	상담 및 프로젝트 관리 화면은 앱, 컨설팅 분석 로직은 서버에서 관리

프로젝트	빌드 및 업데이트 전략
Travel Creator	여행 기록 및 일정 화면은 앱, AI 여행 계획 생성은 서버에서 관리

특히 MSM과 AI 친구 에이전트는 앱과 서버를 분리하는 구조가 중요합니다.

예를 들어 AI 친구의 말투를 수정할 때 앱을 다시 빌드하지 않고 서버의 프롬프트만 수정할 수 있도록 설계하는 것입니다.

이렇게 하면 서비스 개선 속도가 빨라지고 운영 비용도 줄어듭니다.

8. 오늘 반드시 기억할 핵심 정리

핵심 용어	반드시 기억할 내용
Build	소스 코드를 실제 설치 가능한 앱 파일로 만드는 과정
Compile	코드를 기계가 실행할 수 있는 형태로 변환
Package	앱에 필요한 코드와 리소스를 묶는 과정
Signing	앱 개발자를 인증하는 과정
Xcode	iOS 앱을 빌드하는 Mac 전용 도구
Gradle	Android 앱 빌드 도구
Metro	Expo의 JavaScript 번들 생성 도구
EAS Build	Expo 클라우드 앱 빌드 서비스
EAS Update	호환되는 앱에 JavaScript 변경사항 등을 전달하는 서비스
version	사용자에게 보여 주는 앱 버전
buildNumber	스토어가 구분하는 빌드 번호

9. 객관식 퀴즈 정답 및 해설

Q. 스토어에 올린 AI 친구 앱에서 다음 두 가지를 수정합니다.

- (가) 서버에 있는 에이전트 프롬프트를 바꿔 말투를 부드럽게 한다.
- (나) 앱 화면의 전송 버튼 문구를 바꾼다.

정답: 3번 (나)만

해설

- (가) 서버의 프롬프트는 백엔드에서 관리하므로 수정 후 서버에 반영하면 됩니다. 앱을 다시 빌드할 필요가 없습니다.
- (나) 앱 화면 문구는 앱의 JavaScript 코드에 포함되어 있으므로 변경사항을 사용자에게 전달해야 합니다. 새로운 앱 빌드 또는 호환되는 EAS Update가 필요합니다.

따라서 엄밀하게 말하면 (나)는 반드시 재빌드해야 하는 것은 아니지만, 업데이트 배포는 필요합니다.

오늘의 최종 한 줄 정리

빌드는 앱을 제품으로 만드는 과정입니다.

Expo는 Metro로 JavaScript 코드를 묶고 EAS Build로 네이티브 앱을 완성합니다. 그리고 앱과 서버를 분리해 설계하면 불필요한 재빌드를 줄이고 빠르게 서비스를 개선할 수 있습니다.

다음 단계 학습 제안: 지금 배우신 빌드·컴파일·패키징·서명·업데이트의 관계를 AI 친구 에이전트의 실제 개발 구조에 대입해 보면, 모바일 앱의 전체 운영 구조를 훨씬 쉽게 이해하실 수 있습니다.

Chapter03

/강의정리 챕터 03

같은 코드가 폰에 들어가는 세 경로

1. 오늘 강의의 목적

오늘 강의의 핵심은 하나의 Expo 앱을 개발하고, 테스트하고, 다른 사람에게 전달하고, 최종적으로 앱스토어에 출시하는 세 가지 방법을 이해하는 것입니다.

같은 코드를 작성해도 스마트폰에 전달하는 방법은 다릅니다.

오늘 배우는 세 가지 경로는 다음과 같습니다.

1. Expo Go: 개발 중인 앱을 내 폰에서 바로 실행
2. Preview Build: 실제 앱 파일(APK)을 만들어 폰에 직접 설치
3. Production Build: 앱스토어에 제출할 앱 파일을 만들어 정식 출시

이 세 가지의 차이를 이해하면 앱 개발부터 출시까지 전체 흐름을 파악할 수 있습니다.

. 중학생도 이해하는 핵심 개념

① 앱을 스마트폰에 전달하는 세 가지 방법

React Native Tutorial: Build Your First App in 30 Min (2026)

softaims.com

쉽게 자동차에 비유하면 다음과 같습니다.

구분	자동차 비유	실제 의미
Expo Go	운전 연습용 시뮬레이터	개발 중인 앱을 즉시 확인
Preview Build	시험 운행용 완성 자동차	실제 설치 가능한 앱 파일
Production Build	정식 판매용 자동차	앱스토어에 출시할 앱

각 경로를 구체적으로 살펴보겠습니다.

첫 번째: Expo Go

- 노트북에서 개발 서버를 실행합니다.
- 스마트폰의 Expo Go가 개발 서버에 접속합니다.
- Metro가 만든 JavaScript 번들을 받아 실행합니다.
- 코드를 저장하면 화면에 변경사항이 바로 반영됩니다.

장점은 개발 속도가 빠르다는 것입니다.

단점은 개발 서버가 종료되면 실행할 수 없고, 기본적으로 개발 서버와 연결할 수 있는 네트워크 환경이 필요하다는 점입니다.

두 번째: Preview Build

- EAS Build에 앱 빌드를 요청합니다.
- 클라우드에서 실제 앱 파일을 생성합니다.
- APK 설치 링크를 받습니다.
- 링크를 통해 스마트폰에 설치합니다.

이제 노트북을 꺼도 앱을 실행할 수 있습니다.

특히 Android는 APK 파일을 직접 설치할 수 있으므로 친구나 초기 고객에게 테스트용 앱을 전달하기 좋습니다.

세 번째: Production Build

- 정식 출시용 앱 파일을 만듭니다.
- Android는 AAB, iOS는 스토어 배포용 빌드 파일을 준비합니다.
- Google Play 또는 Apple의 앱 배포 절차를 진행합니다.
- 심사를 통과하면 일반 사용자에게 공개할 수 있습니다.

여기서 중요한 차이가 있습니다.

Build는 앱 파일을 만드는 것이고, Submit은 만들어진 앱 파일을 스토어에 제출하는 것입니다.

② 빌드 프로필이란 무엇인가?

핵심 설정 파일: eas.json

빌드 프로필은 앱을 어떤 목적으로, 어떤 형식으로 만들 것인지 정해 놓은 설정입니다.

Expo 프로젝트에서는 `eas.json` 이라는 파일에 이 내용을 작성합니다.

오늘 실습에서 사용하는 프로필은 두 가지입니다.

구분	Preview	Production
목적	내부 테스트	정식 출시
Android 파일	APK	AAB
iOS	내부 테스트용 설치 빌드	스토어 배포용 빌드
설치 방법	설치 링크	스토어 배포 절차
주요 사용자	개발자, 팀원, 테스트 고객	일반 사용자

예를 들어 AI 친구 앱을 만들었다고 가정하겠습니다.

- Preview: 친구 5명에게 앱을 보내서 테스트합니다.
- Production: 테스트 결과를 반영한 앱을 앱스토어에 출시합니다.

즉, Preview는 제품 검증 단계이고 Production은 정식 출시 단계입니다.

③ 앱 서명 키(Signing Key)는 왜 중요한가?

앱 파일에는 개발자가 만든 앱이라는 것을 증명하는 서명이 들어갑니다.

이것은 중요한 문서에 찍는 공식 도장과 같습니다.

예를 들어 처음 출시한 AI 친구 앱이 있다고 가정하겠습니다.

스토어는 처음 제출된 앱의 서명 정보를 기억합니다. 이후 새로운 버전을 제출할 때도 동일한 서명 체계를 사용해 같은 개발자가 업데이트한 앱인지 확인합니다.

따라서 서명 키를 잃어버리거나 다른 사람에게 노출하면 심각한 문제가 발생할 수 있습니다.

Expo EAS는 최초 빌드 과정에서 필요한 서명 키를 생성하고 Expo 계정에 안전하게 보관해 빌드할 때 사용합니다.

기억할 사항은 다음과 같습니다.

- 서명 키는 비밀번호처럼 보호해야 합니다.
- GitHub 저장소에 올리면 안 됩니다.

- `.gitignore`에 키 파일을 제외하도록 설정합니다.
- 키를 분실하거나 유출하지 않도록 관리해야 합니다.

3. 앱 빌드에 들어가는 설정값

① .env와 eas.json의 차이

오늘 강의에서 실무적으로 매우 중요한 내용입니다.

개발할 때는 `.env` 파일에 백엔드 주소를 저장해 사용합니다.

예를 들어 다음과 같습니다.

```
EXPO_PUBLIC_API_URL=https://my-api.example.com
```

그런데 문제가 있습니다.

내 컴퓨터에 있는 `.env` 파일은 일반적으로 GitHub 저장소에 올라가지 않습니다.

따라서 EAS 클라우드가 빌드할 때 이 파일을 자동으로 가져갈 수 없습니다.

그래서 빌드에 필요한 환경변수는 `eas.json`의 각 프로필에 별도로 설정해야 합니다.

개념적으로 다음과 같습니다.

```
{
  "build": {
    "preview": {
      "env": {
        "EXPO_PUBLIC_API_URL": "https://my-api.example.com"
      }
    }
  }
}
```

이 설정이 빠지면 어떤 일이 발생할까요?

- 개발 서버에서는 정상 실행됩니다.
- EAS Build도 성공할 수 있습니다.
- 하지만 설치한 앱에서는 백엔드 주소를 찾지 못해 오류가 발생합니다.

즉, 빌드 성공과 앱의 정상 작동은 서로 다른 문제입니다.

② EXPO_PUBLIC_은 공개해도 되는 값만 사용

환경변수라고 해서 모든 정보를 안전하게 숨길 수 있는 것은 아닙니다.

특히 `EXPO_PUBLIC_`으로 시작하는 변수는 앱의 JavaScript 번들에 포함되므로 사용자가 확인할 수 있습니다.

정보	앱에 포함해도 되는가?
백엔드 API 주소	가능
Supabase 프로젝트 URL	가능
Supabase publishable key	공개 사용을 전제로 보안 규칙을 적용하면 가능
LLM API Secret Key	절대 넣으면 안 됨
Supabase Secret Key	절대 넣으면 안 됨
서버 비밀번호	절대 넣으면 안 됨

이것은 AI 서비스를 만드는 사람에게 특히 중요합니다.

예를 들어 OpenAI API 키를 모바일 앱에 직접 넣으면 앱을 분석하는 사람이 키를 추출해 사용할 수 있습니다.

따라서 다음과 같은 구조가 안전합니다.

How to validate App Store subscriptions on the server-side | by Victoria Seniuk | Medium

medium.com

1. 모바일 앱이 백엔드 서버에 요청합니다.
2. 백엔드 서버가 AI API를 호출합니다.
3. AI가 응답을 생성합니다.
4. 백엔드 서버가 결과를 모바일 앱에 전달합니다.

API 키는 백엔드 서버에서만 관리하는 것이 원칙입니다.

4. 앱 이름과 패키지 이름

앱을 만들 때는 이름도 두 가지를 구분해야 합니다.

구분	설정 위치	역할
앱 이름	`app.json`의 `name`	스마트폰 화면에 표시되는 이름
패키지 이름	`android.package`	Android에서 앱을 구별하는 고유 식별자
번들 ID	`ios.bundleIdentifier`	iOS에서 앱을 구별하는 고유 식별자

예를 들어 AI 친구 앱을 만든다면 다음과 같이 설정할 수 있습니다.

```
{
  "expo": {
    "name": "AI 친구",
    "android": {
      "package": "kr.asp.aifriend"
    },
    "ios": {
      "bundleIdentifier": "kr.asp.aifriend"
    }
  }
}
```

여기서 중요한 것은 패키지 이름과 번들 ID입니다.

- 다른 앱과 중복되지 않아야 합니다.
- 앱을 구별하는 고유한 이름입니다.
- 스토어에 출시한 이후에는 변경을 신중하게 해야 합니다.
- 변경하면 기존 앱의 업데이트가 아니라 새로운 앱으로 취급될 수 있습니다.

따라서 처음 빌드하기 전에 고유한 이름을 정하는 것이 좋습니다.

5. Expo Go와 개발용 빌드의 차이

이번 강의에서는 Expo Go와 Preview Build를 사용하지만, 추가로 알아야 할 개념이 있습니다.

바로 개발용 빌드(Development Build)입니다.

Expo Go에는 Expo가 미리 포함해 둔 네이티브 기능만 들어 있습니다.

그런데 앱을 개발하다 보면 Expo Go에 없는 기능이 필요할 수 있습니다.

예를 들면 다음과 같습니다.

- 특정 결제 SDK
- 외부 로그인 SDK
- 특수한 네이티브 라이브러리
- 일부 푸시 알림 기능

이때 사용하는 것이 개발용 빌드입니다.

개발용 빌드는 내 앱에 필요한 네이티브 기능을 포함해 별도로 만든 개발용 실행 앱입니다.

한 번 설치하면 Expo Go처럼 개발 서버에 연결해 JavaScript 변경사항을 빠르게 확인할 수 있습니다.

세 가지 개발 방식 비교

변경 사항	Expo Go	Development Build	Preview Build
화면 및 문구 변경	즉시 반영	즉시 반영	새 빌드 또는 호환되는 업데이트 필요
네이티브 라이브러리 추가	제한적	재빌드 필요	재빌드 필요
앱 권한 변경	지원 범위 내에서 제한	재빌드 필요	재빌드 필요
노트북 없이 실행	불가능	기본 개발 서버 연결은 필요	가능

여기서 중요한 것은 다음과 같습니다.

- Expo Go: 빠른 개발 테스트
- Development Build: 내 앱에 필요한 네이티브 기능을 포함한 개발 환경
- Preview Build: 실제 사용자에게 전달할 수 있는 테스트용 독립 앱

6. 오늘 강의의 실제 작업 흐름

How to turn every pull request into an instantly installable preview

expo.dev

오늘 실습은 다음 순서로 진행됩니다.

1. Expo Go로 개발 중인 앱을 실행합니다.
2. `app.json`에서 앱 이름과 패키지 이름을 확인합니다.
3. `eas.json`에서 Preview와 Production 프로필을 확인합니다.
4. Preview 프로필에 백엔드 주소를 설정합니다.
5. EAS Build를 실행합니다.
6. 생성된 APK 설치 링크를 받습니다.
7. 스마트폰에 설치하고 실제 실행을 확인합니다.

여기서 특히 확인해야 할 것은 앱 파일이 만들어졌다는 사실보다 설치한 앱이 실제 백엔드와 정상적으로 통신하는지입니다.

7. AI 개발자 및 1인 창업가가 알아야 하는 이유

이번 강의는 실제 제품을 출시하고 운영하는 데 필요한 기초입니다.

특히 다음 네 가지가 중요합니다.

1. 개발과 배포의 분리: 개발용 앱과 실제 고객에게 제공하는 앱을 구분할 수 있습니다.
2. 보안 관리: 앱에 공개해도 되는 정보와 서버에서만 관리해야 하는 정보를 구분합니다.
3. 비용 절감: Expo Go와 EAS Build를 활용해 별도의 복잡한 빌드 환경을 최소화할 수 있습니다.
4. 서비스 운영: 테스트용 앱을 먼저 배포하고 고객 반응을 확인한 뒤 정식 출시할 수 있습니다.

개발자가 아니더라도 이 구조를 이해하면 AI 코딩 도구에 더 정확한 요구사항을 전달할 수 있습니다.

8. 나의 프로젝트에 적용하기

현재 진행하는 프로젝트를 기준으로 적용해 보겠습니다.

프로젝트	적용 방향
MSM	Preview APK로 초기 사용자에게 배포해 글쓰기 및 음성 기록 기능 테스트
AI 친구 에이전트	Preview APK로 대화 기능과 백엔드 연결 검증
ASP 컨설팅	내부 테스트 앱으로 고객별 컨설팅 진행 상황 확인
Travel Creator	여행 일정과 기록 기능을 모바일에서 테스트

특히 MSM은 다음과 같은 구조를 추천합니다.

- 앱: 일기 작성, 사진·음성 입력, 결과물 확인
- 백엔드: 사용자 인증, 데이터 저장, AI 글쓰기 처리
- AI 서버: 자서전 및 웹소설 생성
- EAS Build: 실제 설치 가능한 앱 파일 생성

처음부터 앱스토어에 출시하기보다는 Preview Build를 통해 소수 사용자에게 먼저 제공하고 개선하는 것이 효율적입니다.

9. 오늘 반드시 기억할 핵심 정리

핵심 용어	반드시 기억할 내용
Expo Go	개발 서버와 연결해 앱을 즉시 테스트
Preview Build	실제 설치 가능한 테스트용 앱
Production Build	정식 스토어 출시용 앱

핵심 용어	반드시 기억할 내용
EAS Build	클라우드에서 앱 파일 생성
EAS Submit	빌드된 앱을 스토어에 제출
eas.json	빌드 프로필과 환경변수 설정
Signing Key	앱 개발자를 인증하는 서명 키
.env	로컬 개발 환경변수 파일
EXPO_PUBLIC_	앱에 공개되는 환경변수 접두사
app.json	앱 이름, 권한 등 기본 설정
package	Android 앱의 고유 식별자
bundleIdentifier	iOS 앱의 고유 식별자
Development Build	내 앱 전용 네이티브 개발 환경

10. 객관식 퀴즈 정답 및 해설

정답: 2번

Q. Preview 빌드로 APK를 만들어 폰에 설치했는데, 앱을 열자 "백엔드 주소가 비어 있습니다"라는 오류가 발생했습니다. Expo Go에서는 잘 작동했습니다. 가장 먼저 확인할 곳은?

정답: 2. `eas.json`의 `preview` 프로필 `env`에 `EXPO_PUBLIC_API_URL`이 내 백엔드 주소로 들어 있는지 확인합니다.

해설:

- Expo Go는 로컬 개발 환경의 `.env` 값을 읽어 사용할 수 있습니다.
- 하지만 EAS 클라우드 빌드는 로컬의 `.env`를 자동으로 가져가지 않습니다.
- 따라서 `eas.json`의 Preview 프로필에 필요한 환경변수를 설정해야 합니다.

다른 선택지가 아닌 이유:

- 1번: 와이파이 문제가 아니라 백엔드 주소 자체가 전달되지 않은 상황입니다.
- 3번: `.gitignore`에서 `.env`를 제외하는 것은 정상적인 보안 관리입니다.
- 4번: SDK 버전 문제라면 앱 실행이나 네이티브 호환성 문제가 발생할 가능성이 높습니다.

오늘의 최종 한 줄 정리

같은 코드, 세 가지 배포 경로!

Expo Go는 개발 테스트, Preview Build는 직접 설치와 내부 테스트, Production Build는 정식 출시를 위한 것입니다. 여기에 환경변수, 서명 키, 앱 식별자를 정확하게 관리해야 실제 서비스가 안정적으로 작동합니다.

다음 단계: 오늘 배운 세 가지 배포 경로를 실제 AI 친구 에이전트 프로젝트의 `app.json`, `eas.json`, `.env` 파일에 연결해 보면서 각각의 설정이 어떤 역할을 하는지 확인하시면 좋겠습니다.

Chapter04

/강의정리 챕터 04. 시작 상태 맞추기

핵심 주제: Expo 앱을 실제 휴대폰에 설치하고 실행하기

1. 강의 목적: 개발 환경에서 실제 앱으로 넘어가기

이번 강의의 목적은 지금까지 Expo Go에서 실행하던 AI 친구 에이전트를 독립적으로 실행되는 Android 앱으로 빌드하고 내 휴대폰에 설치하는 것입니다.

앞선 강의에서 앱을 만드는 세 가지 경로를 배웠습니다.

- Expo Go: 개발 서버에 연결해 앱을 빠르게 확인하는 방식
- Preview Build: 실제 휴대폰에 설치해 테스트하는 앱 파일
- Production Build: Google Play나 App Store에 제출하기 위한 앱 파일

이번에는 이 중 Preview Build를 직접 만들어 봅니다.

전체 과정은 다음과 같습니다.

① 개발 환경 확인

Node.js · npm · Expo · 환경변수

② 앱 설정

앱 이름 · 패키지 이름 · 백엔드 주소

③ EAS 클라우드 빌드

Expo 계정 연결 · Android APK 생성

④ 휴대폰에 설치

QR 코드 · APK 다운로드 · 설치

⑤ 독립 실행 확인

노트북 개발 서버를 꺼도 앱 실행 및 대화 가능

. 중학생도 이해하는 핵심 개념

① 앱을 만들기 전에 시작 상태부터 확인합니다

앱을 빌드하려면 기본적인 개발 환경이 준비되어 있어야 합니다.

확인 항목	의미
Node.js 20 이상	JavaScript 프로그램을 실행하는 환경
npm	필요한 프로그램과 라이브러리를 설치하는 도구
node_modules	앱 실행에 필요한 라이브러리 모음
.env	백엔드 서버 주소 등 실행 설정을 보관하는 파일
Expo 계정	Expo 클라우드 빌드 서비스를 사용하기 위한 계정

특히 `.env` 파일에 백엔드 주소가 올바르게 설정되어야 합니다.

예를 들어 다음과 같습니다.

```
EXPO_PUBLIC_API_URL=https://내-staging-서버-주소
```

백엔드 주소가 제대로 작동하는지는 브라우저에서 다음 주소를 열어 확인합니다.

```
https://내-staging-서버-주소/health
```

정상적으로 응답한다면 앱이 통신할 서버가 준비된 것입니다.

② 세 가지 앱 설정값을 정합니다

앱을 빌드하기 전에 다음 설정을 확정해야 합니다.

설정	파일	역할
앱 이름	app.json	휴대폰 홈 화면에 표시되는 이름
패키지 이름	app.json	앱을 구별하는 고유 식별자
백엔드 주소	eas.json	앱이 연결할 서버 주소

예를 들어 다음과 같습니다.

```
{
  "expo": {
```

```
    "name": "하루친구 지수",
    "android": {
      "package": "kr.study.harufriend.jisu"
    },
    "ios": {
      "bundleIdentifier": "kr.study.harufriend.jisu"
    }
  }
}
```

패키지 이름은 앱의 주민등록번호와 비슷합니다.

다른 앱과 구별하기 위한 고유 번호이므로 다른 수강생과 겹치지 않게 정해야 합니다. 특히 스토어에 등록한 뒤에는 변경이 매우 까다롭기 때문에 처음부터 신중하게 결정해야 합니다.

백엔드 주소는 `eas.json`의 `preview`, `production` 프로필에 각각 설정합니다.

- preview: 테스트용 백엔드 주소
- production: 실제 서비스 운영용 백엔드 주소

주소 끝에는 `/`를 붙이지 않습니다.

③ EAS Build는 무엇인가요?

EAS는 Expo Application Services의 약자로, Expo가 제공하는 클라우드 기반 앱 개발·빌드 서비스입니다.

쉽게 말해 내 노트북에서 앱을 직접 조립하지 않고, Expo의 클라우드 컴퓨터에 맡겨 앱 설치 파일을 만드는 서비스입니다.

Create and share internal distribution build - Expo documentation

docs.expo.dev

일반적인 빌드 과정은 다음과 같습니다.

1. 내 컴퓨터에서 앱 소스 코드와 설정을 준비합니다.
2. EAS CLI로 Expo 클라우드에 프로젝트를 전달합니다.
3. Expo 클라우드가 Android 앱을 빌드합니다.
4. 빌드가 완료되면 APK 설치 링크와 QR 코드가 제공됩니다.
5. 휴대폰에서 APK를 내려받아 설치합니다.

실제로 사용하는 명령은 다음과 같습니다.

```
# Expo 계정 로그인
npx eas-cli@latest login
```

```
# 내 Expo 계정 프로젝트와 연결
npx eas-cli@latest build:configure

# Android Preview 앱 빌드
npx eas-cli@latest build --profile preview --platform android
```

여기서 중요한 것은 `npx eas-cli@latest` 입니다.

EAS CLI를 컴퓨터에 별도로 설치하지 않고, 명령을 실행할 때 최신 버전을 사용합니다.

④ 서명 키(Signing Key)는 왜 필요한가요?

서명 키는 앱을 만든 사람이 누구인지 확인하고 앱의 무결성을 검증하는 디지털 신분증과 같습니다.

휴대폰이나 앱스토어는 앱의 서명을 통해 정상적인 앱인지 확인합니다.

이번 실습에서는 EAS가 서명 키를 생성하고 보관하도록 합니다.

- EAS가 키를 생성합니다.
- Expo 계정에 안전하게 보관합니다.
- 내 노트북에는 키 파일을 내려받지 않습니다.
- 다음 빌드에서도 동일한 서명 키를 사용합니다.

서명 키는 앱 업데이트와 배포에서 매우 중요합니다. 키를 잃어버리거나 잘못 관리하면 기존 앱을 정상적으로 업데이트하기 어려워질 수 있습니다.

⑤ Preview Build와 Production Build의 차이

이번 강의에서 반드시 이해해야 할 차이입니다.

구분	Preview Build	Production Build
목적	실제 기기에서 테스트	앱스토어 배포
Android 결과물	APK	AAB
설치 방법	링크 또는 QR 코드	Google Play
백엔드	staging 주소	운영 주소
사용자	개발자·테스터	일반 사용자

APK와 AAB의 차이

- APK: 휴대폰에 직접 설치할 수 있는 앱 파일입니다.

- AAB: Google Play에 제출하는 앱 묶음 파일입니다. Google Play가 사용자 기기에 맞게 설치 파일을 만들어 배포합니다.

따라서 이번 강의에서 만드는 APK는 내 휴대폰이나 팀원 휴대폰에서 테스트하기 위한 파일입니다.

실제 Google Play에 새 앱을 등록할 때는 AAB를 사용합니다.

3. 비유로 이해하기: 앱을 만드는 과정은 자동차 생산과 같습니다

앱 개발 개념	자동차 생산 비유
소스 코드	자동차 설계도
Node.js·npm	생산 설비와 부품
app.json	자동차 모델명과 차대번호
eas.json	생산 방식과 사양 설정
EAS Build	자동차 공장
서명 키	제조사 인증
APK	직접 인도받아 타 볼 수 있는 자동차
AAB	판매점에 공급하는 생산 패키지
휴대폰 설치	자동차를 실제로 인도받아 운행

설계도만 가지고는 자동차를 운전할 수 없습니다.

실제 부품을 조립하고 검사한 뒤 완성된 자동차가 나와야 합니다.

마찬가지로 Expo Go에서 화면이 보이는 것과 실제 앱 설치 파일을 만드는 것은 서로 다른 단계입니다.

4. 실제 실습에서 반드시 확인할 내용

① 빌드 전에 검사하기

다음 명령으로 시작 상태를 확인합니다.

```
node -v
npm -v
npx expo-doctor
```

코딩 에이전트에게는 다음과 같이 요청할 수 있습니다.

```
이 저장소가 Expo 앱을 빌드할 수 있는 상태인지 확인해 줘.

1. node와 npm 버전
2. package.json의 expo 버전
3. .env에 EXPO_PUBLIC_API_URL이 있는지
   (실제 값은 출력하지 마)
4. npx expo-doctor 결과

확인 명령만 실행하고 아무것도 설치하거나 변경하지 마.
```

이 요청에서 중요한 것은 검사만 하고 변경하지 말라고 명확히 지시하는 것입니다.

② 빌드 전에 코드 검사하기

코딩 에이전트에게 빌드를 맡기기 전 다음 두 가지 검사를 먼저 실행하도록 합니다.

```
npx tsc --noEmit
npx expo-doctor
```

- `tsc --noEmit`: TypeScript 코드의 타입 오류를 검사합니다.
- `expo-doctor`: Expo 프로젝트 설정과 의존성의 문제를 검사합니다.

문제가 발견되면 먼저 수정하고 다시 검사합니다.

빌드는 무료 플랜의 월간 횟수 제한이 있으므로 불필요하게 반복하지 않는 것이 좋습니다. 교안에서 제시한 2026년 10월 기준 무료 플랜은 월 30회입니다. 실제 한도와 대기열은 변경될 수 있습니다.

③ 설치한 앱이 정말 독립적으로 작동하는지 확인하기

여기가 이번 실습의 가장 중요한 결과입니다.

1. EAS Build가 완료되면 APK를 휴대폰에 설치합니다.
2. 노트북의 Expo 개발 서버를 종료합니다.
3. 휴대폰에 설치된 앱을 실행합니다.
4. 캐릭터를 만들고 메시지를 보냅니다.
5. AI 친구의 답변이 오는지 확인합니다.

Expo AI Chatbot

노트북의 개발 서버가 꺼져 있어도 앱이 작동한다면 독립 실행에 성공한 것입니다.

왜 가능할까요?

앱을 빌드할 때 화면을 구성하는 코드가 앱 파일 안에 포함되었고, 앱은 인터넷을 통해 백엔드 서버에 직접 요청하기 때문입니다.

즉, 다음 두 가지를 구분해야 합니다.

- 앱 화면 실행: 휴대폰에 설치된 앱 자체가 담당합니다.
- AI 답변 생성: 클라우드에 배포된 백엔드 서버가 담당합니다.

노트북을 꺼도 앱이 작동하는 이유는 화면 코드가 휴대폰에 들어갔기 때문이지, AI 백엔드까지 휴대폰에 들어갔기 때문은 아닙니다.

④ Expo Go와 설치 앱의 데이터는 다릅니다

처음 설치한 앱을 열었을 때 Expo Go에서 만들었던 캐릭터와 대화 기록이 보이지 않을 수 있습니다.

이것은 데이터가 사라진 것이 아닙니다.

Expo Go와 독립 설치 앱은 저장 공간이 서로 다르기 때문에 별개의 사용자처럼 동작할 수 있습니다.

예를 들어 같은 휴대폰에 카카오톡과 다른 메신저 앱이 있다고 생각해 보시면 됩니다. 서로 다른 앱의 내부 저장 공간은 자동으로 공유되지 않습니다.

⑤ 앱 문구를 바꾸면 어떻게 될까요?

Expo Go에서 버튼 문구를 수정하면 개발 서버가 변경된 코드를 전달하므로 바로 반영됩니다.

하지만 독립 설치 앱은 빌드할 때의 코드가 앱 파일 안에 들어 있습니다.

따라서 문구를 바꿔도 기존 설치 앱은 그대로입니다.

- Expo Go: 코드 저장 후 바로 반영
- 독립 설치 앱: 새 빌드 또는 호환되는 EAS Update로 반영
- 백엔드 프롬프트: 서버에서 관리한다면 앱을 다시 빌드하지 않고도 변경 가능

이 차이는 앞으로 모바일 AI 서비스를 운영할 때 매우 중요합니다.

5. iPhone 사용자는 무엇이 다른가요?

Android는 Preview Build로 생성한 APK를 설치 링크나 QR 코드로 내려받아 직접 설치할 수 있습니다.

하지만 iPhone은 Android처럼 APK를 받아 설치하는 방식이 아닙니다.

Mac과 Xcode가 있다면 다음 명령으로 연결된 iPhone에 앱을 설치할 수 있습니다.

```
npx expo run:ios --device
```

이 명령은 기본적으로 Debug 빌드이므로 개발 서버에 연결됩니다.

노트북 없이 실행하려면 다음과 같이 Release 빌드를 사용합니다.

```
npx expo run:ios --device --configuration Release
```

무료 Apple 계정으로 설치한 개발용 앱은 일반적으로 7일 뒤 다시 서명·설치해야 합니다. 개발자 모드 설정도 필요합니다.

따라서 이번 실습에서는 iPhone 사용자는 Expo Go로 확인하고, 실제 설치는 Android 휴대폰을 가진 수강생과 짝을 지어 진행합니다.

6. AI 개발자와 1인 창업가가 알아야 하는 이유

이번 강의는 단순히 앱 파일 하나를 만드는 과정이 아닙니다. AI 서비스를 실제 사용자에게 전달하는 방법을 배우는 과정입니다.

특히 다음 세 가지를 기억해야 합니다.

첫째, 개발 환경과 실제 서비스 환경은 다릅니다.

Expo Go에서 잘 작동한다고 해서 실제 설치 앱에서도 반드시 정상적으로 작동하는 것은 아닙니다. 실제 기기에서 테스트해야 합니다.

둘째, 앱과 백엔드를 분리해야 합니다.

앱에는 화면과 사용자 입력을 처리하는 기능을 넣고, AI 모델 호출과 비밀 API 키는 백엔드에 둡니다.

셋째, 테스트용과 운영용 서버를 구분해야 합니다.

테스트 앱은 staging 서버에, 일반 사용자에게 배포할 앱은 production 서버에 연결해야 합니다.

이렇게 해야 테스트 과정에서 실제 고객 데이터나 운영 서비스에 영향을 주는 일을 줄일 수 있습니다.

7. 나의 프로젝트에 적용하기

이번 강의에서 배운 구조는 AI 친구 에이전트뿐 아니라 앞으로 진행할 프로젝트에도 그대로 활용할 수 있습니다.

프로젝트	모바일 앱에 적용할 기능	백엔드에서 처리할 기능
AI 친구 에이전트	캐릭터 생성, 대화 화면	AI 응답, 기억 관리
My-Story-Maker	글 입력, 이야기 확인	글 분석, 말투 보존, 웹소설 생성
ASP 컨설팅	고객 정보 입력, 프로젝트 현황	컨설팅 분석, 보고서 생성, 프로젝트 관리
Travel Creator	여행 일정 입력, 지도·일정 화면	여행 정보 처리, 일정 추천, AI 분석

예를 들어 MSM을 만든다고 생각해 보겠습니다.

사용자가 휴대폰에서 일기를 입력하면 앱은 입력 내용을 백엔드로 전달합니다. 백엔드는 AI를 이용해 글을 분석하고, 사용자의 말투를 유지하면서 이야기를 생성합니다. 완성된 이야기는 다시 앱에 표시됩니다.

이번 강의에서 배운 앱과 백엔드의 분리 구조가 바로 이런 서비스의 기본 설계입니다.

8. 반드시 기억할 내용

핵심 개념	반드시 기억할 것
Node.js·npm	앱을 실행하고 라이브러리를 관리하는 기본 환경
`.env`	백엔드 주소 등 환경 설정
`app.json`	앱 이름과 고유 식별자
`eas.json`	Preview와 Production 빌드 설정
EAS Build	Expo 클라우드에서 앱 파일 생성
서명 키	앱 제작자와 앱의 무결성을 확인하는 인증 수단
APK	Android 휴대폰에 직접 설치하는 파일
AAB	Google Play에 제출하는 파일
독립 실행	노트북 개발 서버 없이 앱 실행 가능
Expo Go	개발 중 빠르게 확인하는 도구

이번 강의에서 가장 중요한 개념은 다음과 같습니다.

> Expo Go에서 실행되는 앱을 EAS Build로 실제 설치 파일로 만들면, 휴대폰에서 노트북 개발 서버 없이 독립적으로 실행할 수 있습니다. 단, AI 기능은 여전히 백엔드 서버와 통신해야 합니다.

9. 객관식 퀴즈 정답 및 해설

정답: 2번

문제: 설치한 앱의 버튼 문구를 고쳤는데 Expo Go에서는 바뀌고, 휴대폰에 설치한 앱에서는 그대로입니다. 이유는 무엇일까요?

- 1번: 백엔드에서 화면을 받아 온다는 설명이 잘못되었습니다.
- 번: 설치한 앱에는 빌드 당시의 코드가 번들로 들어 있기 때문입니다.
- 3번: Expo Go의 캐시가 설치 앱에 남는 것이 아닙니다.
- 4번: 앱 이름이 달라서 개발 서버에 연결되지 않는 문제도 아닙니다.

설치된 앱은 기존 코드가 들어 있는 별도의 실행 파일입니다. 따라서 변경된 화면 코드를 반영하려면 새로 빌드하거나, 네이티브 호환 범위 내에서 EAS Update를 사용해야 합니다.

10. 한 문장으로 정리

이번 강의는 Expo Go에서 개발하던 AI 친구 에이전트를 EAS Build를 통해 실제 Android 앱으로 만들고, 백엔드와 연결해 노트북 없이도 작동하는 독립 앱으로 전환하는 방법을 배우는 과정입니다.

Chapter05

/강의정리 챕터 05. 스토어 배포 단계

핵심 주제: 테스트 앱을 정식 서비스로 출시하는 과정

1. 강의 목적: 앱을 만드는 것과 출시하는 것은 다릅니다

지난 챕터에서는 Expo Go로 개발하던 AI 친구 에이전트를 EAS Build로 실제 휴대폰에 설치할 수 있는 APK로 만들었습니다.

이번 강의에서는 한 단계 더 나아가 Google Play와 Apple App Store에 앱을 등록하고, 테스트와 심사를 거쳐 일반 사용자에게 공개하는 과정을 배웁니다.

중요한 것은 앱 파일을 만들었다고 바로 스토어에 공개되는 것은 아니라는 점입니다.

전체 흐름은 다음과 같습니다.

① 개발자 계정 등록

Google Play 개발자 계정 / Apple 개발자 계정

② 앱 등록 및 Production 빌드

앱 정보 입력 · AAB / iOS 빌드 생성

③ 테스트 트랙 배포

Google Play 내부·비공개 테스트 / Apple TestFlight

④ 테스트 및 심사

오류 수정 · 정책 확인 · 출시 신청

⑤ 정식 출시

일반 사용자에게 공개

여기서 꼭 기억할 것은 테스트 단계에서 정식 출시 단계로 자동 이동하지 않는다는 점입니다. 개발자가 직접 다음 단계로 진행하고, 필요한 심사를 통과해야 합니다.

. 중학생도 이해하는 핵심 개념

① 개발자 계정이란?

앱스토어에 앱을 올리려면 개인 또는 회사 명의의 개발자 계정이 필요합니다.

구분	Google Play	Apple App Store
개발자 계정	Google Play 개발자 계정	Apple Developer Program
등록 비용	최초 1회 등록비	매년 연회비
앱 등록 관리	Play Console	App Store Connect
테스트 도구	내부·비공개 테스트 등	TestFlight
출시 심사	Google Play 심사	Apple App Review

개발자 계정은 앱을 판매하거나 배포할 수 있는 공식 자격증과 비슷합니다.

계정이 있어야 앱을 등록하고, 테스트하고, 일반 사용자에게 공개할 수 있습니다.

② Google Play의 테스트 단계

android - Internal test track set as inactive on Google Play Console - Stack Overflow

stackoverflow.com

Google Play는 크게 다음과 같은 단계를 거칩니다.

단계	의미	주요 대상
내부 테스트	소수의 등록된 테스터에게 배포	개발자와 팀원
비공개 테스트	초대된 테스터를 대상으로 테스트	초기 사용자 그룹
공개 테스트	조건에 따라 더 넓은 사용자에게 테스트 제공	일반 참여자
프로덕션	정식 출시	일반 사용자

내부 테스트

- 최대 100명의 테스터를 등록할 수 있습니다.
- 등록된 테스터만 참여 링크를 통해 앱을 설치합니다.
- 스토어를 통한 설치 과정을 처음 검증하기에 적합합니다.

비공개 테스트

- 정해진 테스터 그룹을 대상으로 앱을 검증합니다.
- 교안에서 설명한 것처럼 2023년 11월 이후 생성된 개인 개발자 계정에는 추가 출시 조건이 적용됩니다.
- 해당 계정 유형은 12명 이상의 테스터가 14일 연속 비공개 테스트에 참여해야 프로덕션 출시 신청이 가능합니다.

따라서 처음 앱을 등록한 개인 개발자는 하루 이틀 만에 정식 출시까지 진행하기 어려울 수 있습니다.

③ Apple의 TestFlight

TestFlight App Updated With Overhauled Interface for iOS 11 - MacRumors

macrumors.com

Apple은 TestFlight라는 별도의 테스트 서비스를 제공합니다.

두 종류의 테스터가 있습니다.

- 내부 테스터: 개발자 계정 팀에 속한 사람으로, 내부 테스트를 진행합니다.
- 외부 테스터: 팀 외부의 사람으로, 외부 테스트를 위해 베타 심사를 거칠 수 있습니다.

TestFlight는 정식 출시 전에 실제 iPhone 사용자에게 앱을 배포하고 의견을 받을 수 있는 서비스입니다.

예를 들어 AI 친구 앱을 만들었다면 가족이나 지인에게 TestFlight로 앱을 제공해 대화가 자연스러운지, 오류가 없는지 확인할 수 있습니다.

④ 앱 이름과 패키지 이름은 신중하게 정해야 합니다

앞선 챕터에서 배운 패키지 이름과 iOS 번들 ID는 앱을 구별하는 고유 식별자입니다.

한번 스토어에 등록한 앱은 이 식별자를 임의로 바꾸기 어렵습니다.

예를 들어 다음과 같습니다.

```
앱 이름: 하루친구 지수
```

```
Android package:
kr.study.harufriend.jisu

iOS bundleIdentifier:
kr.study.harufriend.jisu
```

앱 이름은 사용자에게 보이는 이름이고, 패키지 이름은 시스템과 스토어가 앱을 식별하는 이름입니다.

스토어에 잘못된 이름이나 식별자를 등록하면 이후 수정과 관리가 복잡해질 수 있으므로 연습용 앱이라도 신중하게 결정해야 합니다.

3. AI 서비스 앱은 일반 앱보다 무엇을 더 준비해야 할까요?

이번 강의에서 가장 중요한 내용 중 하나입니다.

일반적인 앱은 기능이 제대로 작동하는지, 개인정보를 적절하게 관리하는지 등을 심사받습니다.

하지만 AI 에이전트 서비스는 추가로 AI가 만든 콘텐츠, 사용자 대화 데이터, AI의 정체성, 결제 구조까지 고려해야 합니다.

① 개인정보처리방침

개인정보처리방침은 사용자에게 다음 사항을 알려 주는 공개 문서입니다.

- 어떤 데이터를 수집하는가?
- 데이터를 어디에 저장하는가?
- 외부 서비스로 전달하는가?
- 어떤 목적으로 사용하는가?
- 사용자가 데이터를 삭제할 수 있는가?

AI 친구 서비스라면 다음과 같은 데이터가 있을 수 있습니다.

데이터	사용 목적	저장·전달 대상
사용자 대화	AI 응답 및 대화 기록	내 백엔드, 데이터베이스
캐릭터 정보	사용자별 캐릭터 관리	백엔드, Supabase 등
사용자 기억	이전 대화 맥락 유지	백엔드, 데이터베이스
AI 응답 요청	AI 답변 생성	외부 LLM 제공사
계정 정보	사용자 식별	인증 시스템

특히 대화 내용을 외부 LLM API로 보내는 경우에는 개인정보처리방침과 스토어의 데이터 관련 신고 항목에 이를 정확하게 반영해야 합니다.

② 데이터 안전과 개인정보 항목

Google Play에는 데이터 안전(Data Safety) 항목이 있고, Apple에도 앱 개인정보 보호 관련 신고 항목이 있습니다.

예를 들어 사용자가 AI 친구에게 다음과 같이 입력했다고 가정해 보겠습니다.

> 오늘 회사에서 있었던 일을 이야기하고 싶어.

이 대화는 내 백엔드로 전달되고, 다시 외부 LLM 제공사로 전달될 수 있습니다.

사용자는 앱에만 이야기했다고 생각할 수 있지만 실제로는 여러 시스템이 데이터를 처리할 수 있습니다.

따라서 앱 개발자는 데이터가 어디로 전달되는지 정확하게 파악해야 합니다.

③ 계정과 데이터 삭제 기능

사용자가 서비스를 떠나고 싶을 때 자신의 계정과 데이터를 삭제할 수 있어야 합니다.

교안에서는 백엔드에 다음 API가 이미 존재한다고 설명합니다.

```
DELETE /api/data
```

하지만 앱 화면에는 삭제 버튼이 없습니다.

이 경우 백엔드 기능이 있다고 해서 사용자 관점에서 삭제 기능이 완성된 것은 아닙니다.

다음 기능을 앱에 추가해야 합니다.

- 계정 및 데이터 삭제 메뉴
- 삭제 전 사용자 확인
- 삭제 요청 처리
- 삭제 결과 안내

앱에서 데이터를 삭제하는 기능과 별도로 스토어의 계정 삭제 관련 정책도 확인해야 합니다.

④ AI 생성 콘텐츠 신고 기능

AI가 항상 적절한 답변만 하는 것은 아닙니다.

부적절하거나 불쾌한 응답이 생성될 수도 있습니다.

따라서 사용자가 AI 답변을 신고할 수 있는 기능이 필요합니다.

예를 들어 대화 메시지 아래에 다음 버튼을 둘 수 있습니다.

AI 친구의 답변

오늘은 조금 힘든 하루였겠네요. 어떤 일이 있었는지 이야기해 주셔도 괜찮아요.

AI가 생성한 답변

신고하기

신고 기능은 버튼만 만드는 것으로 끝나지 않습니다. 신고 내용을 저장하고, 운영자가 확인하며, 필요한 조치를 할 수 있는 관리 체계도 필요합니다.

⑤ AI임을 명확하게 표시하기

AI 캐릭터가 실제 사람인 것처럼 사용자를 속여서는 안 됩니다.

예를 들어 캐릭터가 사람인 척하기보다는 AI 캐릭터라는 사실을 화면에서 명확히 알려 주는 것이 좋습니다.

이는 사용자에게 서비스의 정체성을 정확하게 전달하고 신뢰를 유지하는 방법입니다.

⑥ 앱 안에서 디지털 상품을 판매하는 경우

이번 퀴즈의 핵심입니다.

AI 친구 서비스에서 크레딧을 판매한다고 생각해 보겠습니다.

예를 들어 사용자가 1,000 크레딧을 구매하면 AI와 더 많은 대화를 할 수 있는 구조입니다.

이때 앱 안에서 디지털 상품을 판매하면 Google Play 또는 Apple의 인앱 결제 정책이 적용될 수 있습니다.

교안에서 제시한 구조는 다음과 같습니다.

권장 구조의 개념

웹 서비스

크레딧 구매 및 결제

모바일 앱

구매한 크레딧 사용

동일한 계정과 백엔드에서 크레딧 잔액 관리

다만 주의할 점이 있습니다. 웹에서 구매하고 앱에서 사용하는 구조라고 해서 모든 경우에 자동으로 정책을 피할 수 있는 것은 아닙니다. 앱에서 외부 결제를 안내하거나 연결하는 방식에도 플랫폼별 제한과 예외가 있으므로 실제 출시 시점의 Google Play 및 Apple 정책을 확인해야 합니다.

4. 비유로 이해하기: 앱스토어 출시는 제품 판매 허가를 받는 과정입니다

앱 개발을 음식점 창업에 비유하면 이해하기 쉽습니다.

앱 출시	음식점 창업
개발자 계정	영업을 위한 사업자 등록
앱 등록	음식점 개업 신고
Production 빌드	판매할 음식 준비
내부 테스트	직원과 지인의 시식
비공개 테스트	초청 고객 시식
스토어 심사	위생 및 영업 기준 확인
정식 출시	일반 고객에게 영업 시작
개인정보처리방침	고객 정보 관리 안내
데이터 삭제 기능	고객 정보 폐기 절차
AI 답변 신고	고객 불만 접수 창구

음식을 만들었다고 바로 음식점을 운영할 수 있는 것은 아닙니다.

위생, 안전, 고객 응대, 결제 등의 준비가 필요합니다.

앱도 마찬가지입니다. 앱을 만드는 기술뿐 아니라 사용자 보호와 운영 체계를 함께 준비해야 합니다.

5. 실습: 출시 준비 점검표 만들기

이번 강의에서는 `docs/store-readiness.md`라는 문서를 만듭니다.

이 문서는 앱스토어에 출시하기 전에 필요한 사항을 하나씩 확인하는 체크리스트입니다.

예시는 다음과 같습니다.

항목	현재 상태	보완할 사항
개인정보처리방침	없음	공개 URL과 문서 작성
데이터 안전 신고	미작성	데이터 수집·전송 경로 조사
계정 및 데이터 삭제	API만 있음	앱에 삭제 메뉴 추가
AI 콘텐츠 신고	없음	메시지별 신고 버튼 추가
AI 표시	있음	화면별 표시 확인
결제 정책	미구현	크레딧 구매 구조 결정

항목	현재 상태	보완할 사항
연령 등급	미작성	실제 서비스 내용에 맞춰 설문 작성

이 문서를 작성하는 목적은 단순히 체크 표시를 하는 것이 아닙니다.

현재 앱이 어디까지 완성되어 있고, 출시 전에 무엇을 더 개발해야 하는지 파악하는 것입니다.

코딩 에이전트 활용하기

이번 강의에서 제시한 프롬프트도 중요합니다.

```
이 앱 코드와 src/lib/api.ts를 읽고,
앱이 사용자에게서 받아 어디로 보내는 데이터를
모두 표로 정리해 줘.

열은 다음과 같이 해 줘.

1. 데이터
2. 받는 화면
3. 보내는 곳(API 경로)
4. 백엔드에서 저장되거나 전달되는 곳

모르는 내용은 모른다고 표시해 줘.

코드는 변경하지 마.
```

여기서 중요한 것은 앱 코드만 보고 백엔드의 데이터 저장 위치까지 추측하지 말라는 점입니다.

앱 코드에서는 API로 데이터를 보내는 사실까지 알 수 있지만, 실제로 백엔드가 그 데이터를 DB에 저장하는지, 외부 AI 서비스로 전달하는지까지는 백엔드 코드를 추가로 확인해야 합니다.

6. AI 개발자와 1인 창업가가 알아야 하는 이유

이번 강의에서 배운 것은 앱을 출시하는 기술보다 더 넓은 의미가 있습니다.

AI 서비스를 사업으로 운영하려면 기술적 완성도와 서비스 운영 책임을 함께 갖춰야 한다는 것입니다.

특히 다음 세 가지가 중요합니다.

1. 개인정보 보호: 사용자의 대화와 기억은 민감한 정보가 될 수 있으므로 수집·저장·전달·삭제 경로를 명확하게 관리해야 합니다.
2. AI 품질 관리: 부적절한 응답을 신고하고 대응할 수 있는 운영 체계가 필요합니다.

3. 수익 모델과 결제 정책: AI 크레딧, 구독, 유료 기능 등 디지털 상품을 어떻게 판매할지 초기 설계부터 고려해야 합니다.

사용자께서 추진하는 ASP 컨설팅 사업에서도 이 부분이 중요합니다. 고객사에 AI 서비스를 제안할 때 단순히 AI 기능을 만드는 데서 끝나지 않고, 개인정보 보호와 운영 정책, 수익 모델까지 함께 설계할 수 있어야 경쟁력 있는 컨설팅이 됩니다.

7. 나의 프로젝트에 적용하기

프로젝트	이번 강의의 적용 내용
AI 친구 에이전트	대화 데이터 보호, AI 신고, 크레딧 결제, 계정 삭제
My-Story-Maker	개인 일기·사진·음성의 개인정보 보호, 원본 데이터 삭제, AI 생성 콘텐츠 표시
ASP 컨설팅	고객사별 개인정보 처리 및 AI 서비스 출시 점검표 제공
Travel Creator	여행자 개인정보, 위치 정보, 예약 정보 관리 및 삭제 기능

특히 MSM은 사용자 개인의 일기와 삶의 이야기를 다루기 때문에 개인정보 보호가 매우 중요합니다.

사용자가 자신의 이야기를 AI에게 맡기는 만큼, 어디에 저장되고 어떤 AI 서비스로 전달되는지 투명하게 알려야 합니다.

나중에 사용자가 서비스를 그만두더라도 자신의 원본 글과 생성된 콘텐츠를 삭제할 수 있도록 설계해야 합니다.

8. 반드시 기억할 내용

개념	핵심 내용
개발자 계정	앱스토어 등록에 필요한 공식 계정
Google Play 내부 테스트	최대 100명의 등록된 테스터 대상
비공개 테스트	제한된 그룹에서 출시 전 검증
TestFlight	Apple의 앱 테스트 서비스
Production Build	스토어 제출용 앱 파일
개인정보처리방침	수집·저장·전달 데이터 공개
데이터 안전	Google Play 데이터 관련 신고
계정 삭제	사용자 계정과 데이터 삭제 지원
AI 신고	부적절한 AI 생성 콘텐츠 신고 기능
인앱 결제	앱 안에서 디지털 상품을 판매할 때 적용되는 결제 정책

9. 객관식 퀴즈 정답 및 해설

정답: 2번

문제: AI 서비스 앱에서 AI 크레딧을 판매하고 싶습니다. 가장 알맞은 구조는 무엇일까요?

- 1번: 앱 안에 웹 결제 페이지를 띄워 판매하는 방식은 외부 결제 유도 정책에 저촉될 수 있습니다.
- 번: 웹 서비스에서 구매하고, 앱에서는 같은 계정의 크레딧을 사용하는 방식입니다.
- 3번: 심사 때만 결제 버튼을 숨기는 것은 정책 준수 방법이 아닙니다.
- 4번: 상품 이름을 후원으로 바꾼다고 결제 정책이 달라지는 것은 아닙니다.

단, 2번도 플랫폼별 외부 결제 관련 정책을 확인한 뒤 구체적인 구현 방식을 결정해야 합니다.

10. 한 문장으로 정리

이번 강의는 앱을 만드는 단계에서 한 걸음 더 나아가, 개발자 계정 등록부터 테스트·심사·정식 출시까지의 스토어 배포 과정과 개인정보 보호, AI 콘텐츠 신고, 결제 정책 등 실제 서비스 운영에 필요한 준비사항을 배우는 과정입니다.

11. 이번 강의에서 다음 단계로 이어갈 실무 과제

이번 강의는 AI 친구 에이전트를 실제 서비스로 출시하기 위한 준비 단계입니다.

우선 `docs/store-readiness.md`를 작성하고, 현재 앱에 없는 기능을 구분하는 것이 좋습니다.

특히 다음 세 가지를 우선순위가 높은 개발 과제로 볼 수 있습니다.

1. 사용자 계정 및 데이터 삭제 기능을 앱 화면과 연결하기
2. AI 답변별 신고 기능과 운영자 확인 체계 만들기
3. 개인정보처리방침과 데이터 전송 경로 정리하기

이 세 가지는 단순한 스토어 심사 통과를 넘어, 실제 고객이 안심하고 사용할 수 있는 AI 서비스를 만드는 기본 조건입니다.

Chapter06

/강의정리 챕터 06. 직접 해 보기의 규칙

핵심 주제: 나만의 앱을 기획하고 Expo로 개발·빌드·설치하기

1. 강의 목적: 배운 기술을 내 앱으로 완성하기

앞선 챕터에서는 AI 친구 에이전트 템플릿을 이용해 Expo 앱을 실행하고, EAS Build로 Android 설치 파일을 만들고, 스토어에 출시하는 과정을 배웠습니다.

이번 강의는 지금까지 배운 내용을 활용해 내가 직접 기획한 앱을 처음부터 만들어 실제 휴대폰에서 실행하는 실습입니다.

여기서 중요한 것은 코딩 실력보다 먼저 서비스의 구조를 스스로 결정하는 것입니다.

- 어떤 문제를 해결하는 앱인가?
- 사용자에게 어떤 화면을 보여 줄 것인가?
- 앱에서 사용하는 데이터는 어디에서 가져오는가?
- 데이터는 어디에 저장하는가?
- 휴대폰의 어떤 기능을 사용할 것인가?
- 실제 설치 파일로 만들어도 정상적으로 작동하는가?

코딩은 AI 코딩 에이전트에게 맡길 수 있습니다. 하지만 무엇을 만들고 어떻게 작동할지 결정하는 일은 개발자의 몫입니다.

. 전체 실습 과정: 4단계로 완성하기

이번 실습은 총 4개 스테이지로 구성됩니다.

1. Stage 0. 앱 계획

   앱의 목적, 화면, 데이터 저장 위치, 기기 기능을 결정합니다.

   설계 단계

2. Stage 1. 화면 만들기

   Expo 프로젝트를 생성하고 필요한 화면을 구현합니다.

   화면 개발

3. Stage 2. 기기 기능 하나 연결하기

   카메라, 위치, 로컬 알림 중 하나를 연결합니다.

   기기 기능 개발

4. Stage 3. 빌드와 설치

   EAS Preview Build로 Android 앱을 만들고 설치합니다.

   실제 앱 완성

5. Stage 4. 출시 준비 점검

   개인정보, 데이터 삭제, AI 콘텐츠, 결제 등 출시 조건을 확인합니다.

   출시 준비

중요한 원칙은 Stage 1~2에서는 Expo Go로만 테스트하고, Stage 3에서 빌드를 한 번만 진행하는 것입니다.

무료 EAS 플랜의 빌드 횟수를 불필요하게 소모하지 않기 위한 전략입니다.

3. Stage 0. 앱 계획하기: 코딩보다 설계가 먼저입니다

이번 강의에서 가장 중요한 단계입니다.

앱을 만들기 전에 `docs/app-plan.md`라는 문서를 작성합니다.

① 앱이 하는 일을 한 문장으로 정의합니다

예를 들어 산책 기록 앱이라면 다음과 같습니다.

> 산책한 날의 사진과 장소를 기록하고, 나중에 목록으로 다시 확인하는 앱이다.

이 한 문장으로 앱의 목적이 명확해집니다.

② 필요한 화면을 정합니다

처음부터 많은 화면을 만들 필요는 없습니다.

화면	역할
기록 목록	과거 산책 기록 확인
새 기록	사진과 장소 입력
기록 보기	저장된 산책 기록 상세 확인

하루 안에 완성하려면 화면을 3개 이하로 제한하는 것이 좋습니다.

③ 데이터가 어디에서 오고 어디에 저장되는지 정합니다

여기가 가장 중요한 설계 결정입니다.

데이터 저장 방식은 크게 두 가지입니다.

방식 A. 휴대폰 내부에 저장

Why ToolBerry Is Offline-First | ToolBerry Blog

toolberry.net

- AsyncStorage 등으로 휴대폰에 데이터를 저장합니다.

- 별도 백엔드 서버가 필요하지 않습니다.
- 인터넷 없이도 기본 기능을 사용할 수 있습니다.
- 앱을 삭제하거나 휴대폰을 바꾸면 데이터가 사라질 수 있습니다.

방식 B. 서버에 저장

What is offline-first & why does it matter? - onbalance

onbalance.app

- 앱에서 서버로 데이터를 전송합니다.
- 백엔드와 데이터베이스가 필요합니다.
- 다른 기기에서도 같은 데이터를 사용할 수 있습니다.
- 사용자 인증과 데이터 관리가 필요합니다.

쉽게 말하면 휴대폰 내부 저장은 개인 수첩이고, 서버 저장은 인터넷에 연결된 공동 보관함입니다.

구분	휴대폰 내부 저장	서버 저장
서버 필요	아니요	예
인터넷 필요	기본 저장·조회는 불필요	일반적으로 필요
다른 기기에서 사용	어려움	가능
앱 삭제 시 데이터	함께 사라질 수 있음	서버에 남을 수 있음
개발 난이도	상대적으로 낮음	상대적으로 높음

하루 안에 앱을 완성하려면 휴대폰 내부 저장이나 이미 구축된 백엔드를 사용하는 것이 좋습니다.

④ 휴대폰 기능을 하나 선택합니다

이번 실습에서는 다음 세 가지 중 하나를 선택합니다.

기능	Expo 라이브러리	활용 예
카메라·사진	`expo-image-picker`	영수증 촬영, 산책 사진
위치	`expo-location`	산책 장소, 여행 위치
로컬 알림	`expo-notifications`	약속 알림, 유통기한 알림

주의할 점은 모든 기기 기능이 Expo Go에서 동일하게 지원되는 것은 아니라는 것입니다.

라이브러리 문서에서 `Included in Expo Go` 표시를 확인해야 합니다.

4. Stage 1. 화면 만들기: Expo 프로젝트 생성

새로운 앱 프로젝트를 만들 때 사용하는 명령은 다음과 같습니다.

```
npx create-expo-app@latest my-first-app
```

프로젝트 폴더로 이동합니다.

```
cd my-first-app
```

필요한 라이브러리를 설치하고 실행합니다.

```
npm install
npx expo start
```

휴대폰의 Expo Go로 QR 코드를 찍으면 개발 중인 앱을 확인할 수 있습니다.

Expo Go vs Development Builds: Which should you use?

expo.dev

반드시 확인해야 하는 검사

```
npx tsc --noEmit
npx expo-doctor
```

- `tsc --noEmit`: 코드에 타입 오류가 있는지 확인합니다.
- `expo-doctor`: Expo 프로젝트의 설정과 의존성에 문제가 있는지 확인합니다.

두 검사 모두 오류 없이 끝나야 다음 단계로 진행합니다.

5. Stage 2. 기기 기능 하나 연결하기

이번 단계에서는 단순한 화면을 넘어 휴대폰의 실제 기능을 사용합니다.

예를 들어 산책 기록 앱에서 카메라를 선택했다면 다음과 같은 흐름이 필요합니다.

새 산책 기록 화면

사진 촬영 또는 선택 버튼

카메라 권한 요청

허용

사진 촬영 진행

거부

사진 없이 기록 진행

여기서 반드시 기억할 원칙이 있습니다.

사용자가 권한을 거부해도 앱 전체가 멈추면 안 됩니다.

카메라 권한을 거부했다면 사진 촬영 기능만 사용할 수 없도록 하고, 나머지 기록 기능은 정상적으로 작동해야 합니다.

이것을 예외 상황을 고려한 앱 설계라고 합니다.

6. Stage 3. 빌드와 설치: 실제 앱으로 만들기

Stage 1과 Stage 2에서 문제가 없는지 충분히 확인한 다음 빌드를 진행합니다.

```
npx eas-cli@latest build --profile preview --platform android
```

빌드가 완료되면 설치 링크나 QR 코드를 통해 Android 휴대폰에 APK를 설치합니다.

이번 단계의 성공 조건은 다음과 같습니다.

- Expo Go에서 보이던 화면이 설치 앱에서도 보입니다.
- 카메라·위치·알림 중 선택한 기능이 정상적으로 작동합니다.
- 노트북의 Expo 개발 서버를 종료해도 앱이 실행됩니다.
- 데이터가 정상적으로 저장되고 다시 조회됩니다.

가장 중요한 성공 기준

노트북을 닫고 개발 서버를 꺼도 휴대폰에 설치한 앱이 독립적으로 실행되어야 합니다.

이것이 단순한 개발 테스트와 실제 배포 가능한 앱의 차이입니다.

7. Stage 4. 출시 준비 점검표 만들기

지난 챕터 5에서 배운 `docs/store-readiness.md`를 이번에 만드는 앱에 맞게 작성합니다.

점검 항목은 다음과 같습니다.

점검 항목	확인할 내용
개인정보처리방침	수집하는 데이터와 처리 방식 공개

점검 항목	확인할 내용
데이터 안전	외부 전송 및 저장 데이터 신고
계정·데이터 삭제	사용자 데이터 삭제 기능
AI 콘텐츠 신고	AI 기능이 있다면 부적절한 콘텐츠 신고
AI임을 밝히기	AI 서비스를 사람으로 오인하지 않도록 표시
결제 정책	디지털 상품 판매 방식 확인
연령 등급	실제 콘텐츠에 맞게 등급 설문 작성

각 항목에 반드시 `있음` 또는 `없음`을 적습니다.

그리고 비어 있는 항목 중 하나를 실제로 개발하거나 보완합니다.

이 과정은 앱이 완성되었다는 사실을 확인하는 데서 끝나는 것이 아니라, 일반 사용자에게 제공할 수 있는 수준으로 개선하는 과정입니다.

8. 비유로 이해하기: 앱 개발은 작은 가게를 여는 과정입니다

앱 개발 단계	가게 창업 비유
앱 계획	어떤 가게를 열지 결정
화면 설계	매장 내부 구조와 메뉴판 설계
기기 기능	계산대·카메라·호출벨 설치
데이터 저장	고객 주문과 매출 기록 관리
Expo Go 테스트	가게를 열기 전 시범 운영
Preview Build	실제 매장 개점
출시 준비 점검	안전·위생·영업 준비 확인

아무리 좋은 기술이 있어도 무엇을 만들지 정하지 못하면 완성하기 어렵습니다.

따라서 이번 강의는 코드를 잘 작성하는 방법보다는 작은 서비스 하나를 기획하고 끝까지 완성하는 경험에 더 큰 의미가 있습니다.

9. AI 개발자와 1인 창업가가 알아야 하는 이유

이번 강의는 앞으로 AI 서비스를 직접 만들고 사업화하려는 사람에게 매우 중요한 연습입니다.

특히 다음 세 가지를 배울 수 있습니다.

1. 기획 역량: 문제를 정의하고 화면과 데이터 구조를 설계하는 능력
2. AI 협업 역량: 코딩 에이전트에게 명확한 요구사항을 주고 결과를 검증하는 능력

3. 제품 완성 역량: 화면 개발에서 끝나지 않고 실제 기기에 설치해 독립 실행하는 능력

코딩 에이전트가 코드를 만들어 주더라도 앱의 목적과 구조를 이해하지 못하면 결과물을 제대로 평가하기 어렵습니다.

이번 실습에서 앱 계획을 먼저 작성하도록 한 이유가 바로 여기에 있습니다.

10. 나의 프로젝트에 적용하기

사용자께서 진행하는 프로젝트에 연결하면 이번 실습의 의미가 더 명확해집니다.

프로젝트	이번 실습에서 적용할 수 있는 작은 앱
My-Story-Maker	일기 입력 및 사진 첨부 앱
ASP 컨설팅	고객 미팅 기록 및 상담 메모 앱
Travel Creator	여행 장소와 사진 기록 앱
AI 친구 에이전트	AI 캐릭터와 대화하는 모바일 앱

특히 **MSM(My-Story-Maker)**에 적용한다면 다음과 같은 작은 MVP를 추천합니다.

MSM 미니 앱: 나의 하루 기록하기

写真も貼れるシンプルな日記アプリ、Nikkyをリリースしました！ - AppSeedのアプリ開発ブログ

develop.hateblo.jp

화면	기능
기록 목록	이전에 작성한 일기 확인
새 기록	오늘의 이야기와 사진 입력
기록 상세	작성한 일기와 사진 확인

데이터 저장: 처음에는 휴대폰 내부 저장

기기 기능: 사진 선택

확장 방향: 이후 Supabase와 AI 백엔드를 연결해 일기를 분석하고 사용자의 말투를 유지한 이야기로 변환

이렇게 하면 처음부터 MSM 전체를 만들 필요 없이, 작은 앱부터 시작해 점차 확장할 수 있습니다.

11. 주관식 퀴즈 정답 가이드

Q1. Expo Go와 설치 앱의 권한 창 문구가 다른 이유는?

핵심: 앱별 권한 설명 설정

Expo Go는 Expo Go라는 개발용 앱의 설정을 사용하지만, 독립 설치 앱은 내가 만든 앱의 설정을 사용하기 때문입니다.

카메라나 위치 등의 권한 설명 문구는 앱의 설정 파일인 `app.json` 또는 `app.config.js`에서 설정합니다.

예를 들어 카메라 권한이라면 다음과 같이 작성할 수 있습니다.

```
{
  "expo": {
    "plugins": [
      [
        "expo-image-picker",
        {
          "cameraPermission": "산책 기록에 사용할 사진을 촬영하기 위해 카메라 접근 권한이 필요합니
다.",
          "photosPermission": "산책 기록에 사진을 첨부하기 위해 사진 접근 권한이 필요합니다."
        }
      ]
    ]
  }
}
```

여기서 중요한 점은 다음과 같습니다.

- 사용자가 권한을 왜 허용해야 하는지 구체적으로 설명합니다.
- 실제 앱 기능과 관련된 권한만 요청합니다.
- 권한 설명을 변경하면 네이티브 설정이 바뀌므로 새 빌드가 필요합니다.

300자 이내 답변 예시

> Expo Go는 Expo Go 자체의 권한 설정을 사용하지만, 독립 설치 앱은 내가 만든 앱의 설정을 사용하기 때문에 문구가 다릅니다. 권한 설명은 app.json의 플러그인 설정에서 정합니다. 카메라나 사진 접근이 왜 필요한지 실제 기능과 연결해 구체적으로 설명하고, 문구를 변경하면 새 빌드로 반영해야 합니다.

Q2. 직접 해 보기 회고

이 문제는 정답을 맞히는 퀴즈가 아니라 문제 해결 과정을 돌아보는 질문입니다.

다음 순서로 작성하면 됩니다.

1. 무엇에서 막혔는가?
2. 처음에는 왜 문제가 생겼다고 생각했는가?

3. 어떤 로그나 문서를 확인했는가?
4. 실제 원인은 무엇이었는가?
5. 어떻게 해결했는가?

작성 예시

> Expo Go에서는 정상 작동했지만 설치 앱에서 카메라 권한 문구가 다르게 표시되었습니다. 처음에는 앱 코드의 문제라고 생각했지만 Expo 설정 문서를 확인해 권한 설명이 앱별 네이티브 설정에서 관리된다는 사실을 알았습니다. app.json의 권한 문구를 수정하고 새로 빌드해 해결했습니다.

실제 실습에서 겪은 문제를 기준으로 작성하시면 됩니다.

12. 반드시 기억할 내용

핵심 개념	기억할 내용
앱 계획	코딩 전에 목적·화면·데이터 구조를 결정
화면 설계	하루 안에 끝내려면 3개 이하 권장
데이터 저장	휴대폰 내부 저장과 서버 저장의 차이 이해
기기 기능	카메라·위치·알림 중 하나 선택
권한 처리	사용자가 거부해도 앱이 멈추지 않도록 설계
Expo Go	개발 단계에서 화면과 기능 테스트
TypeScript 검사	`npx tsc --noEmit`
Expo 검사	`npx expo-doctor`
EAS Build	마지막 단계에서 한 번만 빌드
출시 준비	`store-readiness.md` 작성 및 보완

13. 한 문장으로 정리

이번 강의는 코딩 에이전트와 협업하되 앱의 목적·화면·데이터 구조는 직접 설계하고, Expo Go에서 개발한 앱을 실제 휴대폰에 설치할 수 있는 독립 실행 앱으로 완성하는 실전 프로젝트입니다.

14. 이번 강의의 핵심 교훈

이번 챕터에서 가장 중요하게 생각해야 할 것은 다음 문장입니다.

AI가 코드를 대신 작성해 줄 수는 있지만, 무엇을 만들고 어떻게 작동해야 하는지를 결정하는 것은 나의 역할입니다.

이는 앞으로 ASP 컨설팅이나 MSM 같은 서비스를 개발할 때도 동일합니다.

좋은 AI 개발자는 코드를 많이 작성하는 사람이 아니라, 고객의 문제를 정확하게 정의하고 AI가 구현할 수 있는 구조로 바꾸며, 완성된 결과물이 실제로 작동하는지 검증하는 사람입니다.

Chapter07

/강의정리 챕터 07. 모바일 앱 개발부터 스토어 출시까지 전체 정리

핵심 주제: 개발 → 빌드 → 설치 → 테스트 → 스토어 배포

이번 챕터는 모바일 앱 과정에서 배운 내용을 정리하고, 처음에 이해했던 Expo Go와 실제 설치 앱의 차이를 직접 경험한 결과와 비교하는 복습 과정입니다.

가장 중요한 교훈은 개발 중인 앱을 실행하는 것과 실제 사용자가 설치해서 사용하는 앱을 만드는 것은 서로 다른 과정이라는 점입니다.

1. 오늘 배운 것: 전체 흐름 이해하기

모바일 앱 개발과 배포에는 크게 세 가지 단계가 있습니다.

① 개발 실행: Expo Go

개발 서버에서 코드를 받아 휴대폰에서 실행합니다.

- 코드를 수정하면 빠르게 반영됩니다.
- 개발 서버와 네트워크 연결이 필요합니다.
- 개발 중 화면과 기능을 확인하는 데 적합합니다.

② 앱 빌드: EAS Build

클라우드에서 실제 설치 가능한 앱 파일을 만듭니다.

- 앱 코드가 설치 파일에 포함됩니다.
- 노트북의 개발 서버 없이 실행할 수 있습니다.
- APK 링크나 QR 코드로 다른 사람에게 배포할 수 있습니다.

③ 스토어 제출 및 출시

개발자 계정으로 앱을 등록하고 테스트와 심사를 진행합니다.

- Google Play와 Apple App Store에 등록합니다.
- 개인정보와 결제 정책 등을 확인합니다.
- 필요한 심사를 통과하고 출시를 승인해야 공개됩니다.

세 단계의 핵심 차이

구분	Expo Go	EAS Build	스토어 배포
목적	개발·테스트	설치 파일 제작	일반 사용자에게 공개
코드 제공	개발 서버	앱 내부 번들	앱 내부 번들
노트북 개발 서버	필요	불필요	불필요
수정 반영	빠르게 반영	재빌드 또는 EAS Update	업데이트 배포 필요
사용 대상	개발자	테스터·팀원	일반 사용자

여기서 한 가지 추가로 기억할 내용이 있습니다.

앱의 화면 문구나 JavaScript 코드는 호환되는 EAS Update로도 업데이트할 수 있습니다. 다만 카메라 권한 설정, 네이티브 라이브러리 등 기기 내부 구조가 바뀌면 새로운 빌드가 필요합니다.

. 한 번 정하면 바꾸기 어려운 것: 패키지 이름과 서명 키

앱을 출시하기 전에 반드시 신중하게 결정해야 할 두 가지가 있습니다.

① 패키지 이름(Bundle ID)

패키지 이름은 앱을 구별하는 고유 식별자입니다.

예를 들어 다음과 같습니다.

```
kr.study.harufriend.jisu
```

Android에서는 `android.package`, iOS에서는 `ios.bundleIdentifier`로 설정합니다.

이 값은 앱스토어가 기존 앱과 새로운 앱을 구분하는 기준이므로 최초 등록 전에 확정하는 것이 중요합니다.

② 서명 키(Signing Key)

서명 키는 앱이 정당한 개발자에 의해 만들어졌다는 것을 확인하는 디지털 인증 수단입니다.

쉽게 말하면 앱의 신분증과 같습니다.

이번 강의에서는 EAS가 서명 키를 생성하고 보관하도록 했습니다.

- 키 파일을 노트북에 내려받지 않습니다.
- GitHub 저장소에 올리지 않습니다.
- `.gitignore`를 통해 키 파일이 커밋되지 않도록 관리합니다.
- 이후 업데이트에서도 기존 앱과 동일한 서명 체계를 유지합니다.

핵심은 앱의 고유 식별자와 서명 키를 처음부터 안전하게 관리하는 것입니다.

3. 앱 출시 준비: 개발만 잘해서는 부족합니다

AI 에이전트 앱은 일반적인 앱보다 더 많은 운영 준비가 필요합니다.

스토어 출시 전 확인해야 할 주요 항목은 다음과 같습니다.

점검 항목	필요한 이유
개인정보처리방침	사용자 데이터를 어떻게 처리하는지 알림
데이터 안전	수집·저장·외부 전송 데이터 신고
계정 및 데이터 삭제	사용자가 자신의 데이터를 삭제할 수 있도록 지원
AI 콘텐츠 신고	부적절한 AI 응답을 신고할 수 있도록 지원
AI 표시	AI를 사람으로 오인하지 않도록 안내
앱 내 결제	디지털 상품 판매 시 스토어 결제 정책 준수
연령 등급	앱의 실제 콘텐츠에 맞는 등급 설정

특히 AI 친구 서비스는 사용자 대화가 백엔드와 외부 LLM 제공사로 전달될 수 있으므로 데이터 흐름을 정확하게 파악해야 합니다.

그리고 테스트 트랙에 앱을 올렸다고 자동으로 정식 출시되는 것은 아닙니다.

사람이 다음 단계로 진행하고 필요한 심사를 통과해야 일반 사용자에게 공개됩니다.

4. 내 앱 만들기: 기획이 코딩보다 먼저입니다

이번 과정에서 직접 앱을 만들면서 배운 가장 중요한 원칙입니다.

코딩 에이전트에게 구현을 맡기더라도 앱의 구조는 내가 먼저 결정해야 합니다.

앱을 만들기 전에 다음 사항을 정합니다.

1. 앱이 해결하려는 문제
2. 사용자에게 보여 줄 화면 목록

3. 데이터가 어디에서 오는지
4. 데이터가 어디에 저장되는지
5. 카메라·위치·알림 중 어떤 기기 기능을 사용할지

예를 들어 산책 기록 앱이라면 다음과 같이 설계할 수 있습니다.

구분	설계 내용
목적	산책 사진과 장소 기록
화면	기록 목록, 새 기록, 기록 상세
데이터	휴대폰 내부 저장
기기 기능	카메라와 위치
개발 환경	Expo Go
배포	EAS Preview Build

이렇게 먼저 설계하면 코딩 에이전트에게도 명확한 지시를 할 수 있고, 개발 과정에서 불필요한 수정이 줄어듭니다.

5. 비유로 이해하기: 개발과 출시는 자동차 생산과 판매

모바일 앱	자동차
Expo Go	설계 중 시제품을 시험 운전
소스 코드	자동차 설계도
EAS Build	자동차 공장에서 완제품 생산
APK	직접 인도받아 운전할 자동차
서명 키	제조사 인증
스토어 심사	판매 전 안전·품질 검사
앱스토어 출시	일반 고객에게 판매 시작
앱 업데이트	자동차 개선 모델 출시

시제품이 정상적으로 움직인다고 완제품도 반드시 정상이라는 보장은 없습니다.

실제 사용자에게 제공하기 전에 독립적으로 실행되는지, 데이터가 정상적으로 저장되는지, 보안과 안전에 문제가 없는지 확인해야 합니다.

6. AI 개발자와 1인 창업가가 알아야 하는 이유

이번 강의에서 배운 것은 단순한 모바일 앱 개발 기술이 아닙니다.

AI 서비스를 실제 사업으로 연결하기 위한 제품 개발의 전체 흐름입니다.

특히 다음 세 가지 역량이 중요합니다.

- 서비스 기획 역량: 사용자 문제를 정의하고 화면과 데이터 구조를 설계하는 능력
- AI 협업 역량: 코딩 에이전트에게 정확한 요구사항을 전달하고 결과를 검증하는 능력
- 제품 출시 역량: 개발 결과물을 실제 사용자에게 전달하고 운영할 수 있도록 준비하는 능력

50대 비개발자 창업가에게도 중요한 변화입니다.

모든 코드를 직접 작성하지 않더라도 기술의 구조를 이해하고 개발 과정을 관리할 수 있다면 AI를 활용해 자신의 서비스를 직접 만들어 갈 수 있습니다.

7. 나의 프로젝트와 연결하기

이번에 배운 모바일 앱 개발·배포 구조는 추진 중인 프로젝트에 공통으로 활용할 수 있습니다.

프로젝트	모바일 앱의 역할	백엔드의 역할
AI 친구 에이전트	캐릭터와 대화	AI 응답과 기억 관리
My-Story-Maker	일기·사진·음성 입력	글 분석과 이야기 생성
ASP 컨설팅	고객 정보와 프로젝트 관리	AI 분석과 컨설팅 보고서 생성
Travel Creator	여행 일정과 사진 기록	여행 정보 검색과 AI 일정 추천

특히 MSM은 모바일 앱에서 일기와 사진을 입력하고, 백엔드에서 이를 분석해 사용자의 말투를 유지한 이야기로 만드는 구조가 적합합니다.

따라서 이번 강의에서 배운 앱과 백엔드의 분리, 데이터 저장, 실제 빌드와 설치 경험은 MSM을 구현하는 데 직접 활용할 수 있습니다.

8. 주관식 퀴즈: 과거의 이해와 실제 경험 비교하기

핵심: 알고 있던 것과 직접 경험한 것의 차이

질문

섹션 1에서 Expo Go로 실행하는 앱과 스토어에서 받은 앱의 차이를 적었습니다. 그 답을 다시 읽고, 오늘 직접 본 것과 비교해 맞았던 것과 새로 알게 된 것을 적어 보세요.

이 문제는 단순히 개념을 설명하는 문제가 아닙니다. 처음에 이해했던 내용을 실제 경험을 통해 검증하는 회고 문제입니다.

① 처음에 알고 있던 내용과 실제 경험 비교

항목	처음에 이해했던 내용	직접 경험해서 확인한 내용
앱 실행	Expo Go는 개발 서버가 필요함	설치 앱은 노트북 없이 실행됨
코드 수정	Expo Go는 수정 사항이 바로 반영됨	설치 앱은 기존 코드가 유지됨
데이터 저장	앱마다 데이터가 다를 수 있음	Expo Go와 설치 앱이 별도 저장 공간을 사용해 새 게스트로 시작함
배포	앱 파일을 만들면 배포 가능	스토어 등록, 테스트, 심사, 승인 절차가 필요함
앱 업데이트	코드를 수정하면 앱이 바뀜	재빌드 또는 호환되는 EAS Update 등 별도 배포가 필요함

② 300자 이내 답변 예시

처음에는 Expo Go와 설치 앱의 차이가 개발 서버 연결 여부와 코드 반영 속도라고 이해했습니다. 직접 실습해 보니 설치한 앱은 노트북의 개발 서버를 꺼도 실행되고, 코드를 수정해도 기존 앱에는 반영되지 않는다는 것을 확인했습니다. 또한 Expo Go와 설치 앱은 저장 공간이 달라 새 게스트로 시작했습니다. 스토어 배포도 파일 업로드로 끝나는 것이 아니라 테스트와 심사를 거쳐야 한다는 점을 새롭게 배웠습니다.

이 답변의 핵심은 네 가지입니다.

1. 처음 알고 있던 내용 중 맞았던 것
2. 노트북 없이 앱이 실행된 경험
3. 코드 수정이 기존 설치 앱에 반영되지 않은 경험
4. 새 게스트와 스토어 배포 절차를 통해 새롭게 알게 된 점

9. 반드시 기억할 핵심 내용

핵심 개념	한 줄 정리
Expo Go	개발 서버에 연결해 빠르게 테스트
EAS Build	클라우드에서 실제 설치 파일 생성
스토어 배포	테스트와 심사를 거쳐 일반 사용자에게 공개
패키지 이름	앱을 구별하는 고유 식별자
서명 키	앱의 신뢰성과 업데이트를 위한 인증 수단
데이터 분리	앱과 백엔드가 각각 담당하는 역할 구분
출시 준비	개인정보·삭제·AI 신고·결제 정책 확인
앱 기획	코딩 전에 화면과 데이터 구조부터 결정

10. 한 문장으로 정리

Expo Go로 개발한 앱을 EAS Build로 독립 실행 가능한 설치 파일로 만들고, 실제 사용자에게 배포하려면 테스트·보안·개인정보·결제 정책까지 준비해야 한다는 것이 이번 모바일 앱 과정의 핵심입니다.

11. 이번 과정에서 얻은 가장 중요한 교훈

이번 모바일 앱 과정은 기술을 배우는 것에서 제품을 완성하는 단계로 넘어가는 경험이었습니다.

앞으로 AI 서비스를 만들 때는 다음 순서를 기억하시면 좋겠습니다.

문제 정의 → 앱 기획 → 화면 설계 → 데이터 설계 → AI 코딩 에이전트 활용 → 테스트 → 빌드 → 배포 → 운영

코딩은 이 과정 중 하나일 뿐입니다.

실제 사업에서 더 중요한 것은 고객이 어떤 문제를 겪고 있는지 이해하고, 그 문제를 해결하는 서비스를 설계하고, 사용자가 실제로 사용할 수 있는 제품으로 완성하는 능력입니다.